

Security Analysis of Master-Password-Protected Password Management Protocols

Yihe Duan, Ding Wang, Yanduo Fu

College of Cyber Science, Nankai University, Tianjin 300350, China; wangding@nankai.edu.cn

Key Laboratory of Data and Intelligent System Security (NKU), Ministry of Education, Tianjin 300350, China

Tianjin Key Laboratory of Network and Data Security Technology, Nankai University, Tianjin 300350, China

Abstract—Password managers (PMs) are useful tools that help users manage their login credentials, alleviating the burden of memorizing an ever-increasing number of passwords. Master-password-protected password management (M3PM) protocols characterize the interaction between the client and the PM’s server. In this protocol, the client uses the master password for authentication, and the server assists in retrieving credentials across devices. Given the ongoing PM data breaches and users’ concerns about potential server misuse, it is crucial for the server to remain oblivious to both the master password and the credentials. The pivotal role of M3PM protocols underscores the need for a systematic and formal security analysis.

In this paper, we, *for the first time*, present an extensive formal analysis of M3PM protocols. We identify the de facto M3PM protocols from 43 PMs in industry and academia by defining a methodology that includes documentation analysis, traffic analysis, and reverse engineering. To formalize the security properties of M3PM protocols, we propose a set of ideal functionalities within the universal composability (UC) framework. We categorize offline guessing attacks on master passwords into four types based on the knowledge of the adversary. Our analysis shows that 38 of the 43 PMs are vulnerable to at least one type of offline guessing attack, demonstrating the circumstances under which various M3PM protocols with single master password protection fail to resist such attacks. Additionally, we identify an oracle attack where a corrupted server can learn the encryption key of the well-known open-source Passbolt, and demonstrate that 1Password’s dual-key mechanism provides strong protection for users’ master passwords and credentials.

1. Introduction

Passwords stubbornly remain the most prevalent authentication method and are unlikely to be replaced in the foreseeable future [2], [4], [16], [17]. The average number of accounts that users need to maintain has increased to 80-168 [26], [40], [43]. This brings great burdens for users to manage (e.g., create/remember/type) passwords. In response, users often adopt vulnerable behaviors such as choosing popular passwords (e.g., 123456 and password) [55], reusing passwords [42], and creating passwords containing personally identifiable information (PII) [54], which makes password vulnerable to damaging password guessing attacks

[34], [42], [55], [56]. Besides, there are usability issues, e.g., typos often occur when typing passwords, especially on mobile devices. In a nutshell, managing a large number of passwords is a great challenge for common human users.

Password managers (PMs), tools for helping users manage credentials, are promising to alleviate this issue. Benefiting from the free of human memorization, users can generate strong (e.g., long and containing various characters) and unique passwords for each credential by PMs. Users can *store* their credentials into PMs, and unlock the PMs to *get* their passwords when they need. PMs help users avoid vulnerable behaviors, and reduce the cognitive burden caused by memorizing multiple passwords, striking a balance between password security and usability [21]. Therefore, PMs are highly recommended by standard bodies (e.g., NIST [3] and NCSC [1]) and security experts [27], [36], [61].

The security of PMs, given their centralization of sensitive user credentials, has become a serious concern for both industry [46], [52] and academia [23], [24], [33], [41], [50]. LastPass [6] experienced data breaches three times within past three years [8], [51]. While LastPass asserts that no passwords were compromised in 2022, this breach has recently been found disastrous and linked to a theft of \$4.4 million in cryptocurrency [8]. Furthermore, potential security concerns have been identified as deterrents for users in adopting PMs [9], [35], [39], [44], [45]. Users not only fear data breaches in PMs but also exhibit limited trust in entrusting all their credentials to a third party [9], [44]. To instill more trust, several PMs assert that they are “zero-knowledge” regarding users’ password vaults [11], [31]. This term implies that the PMs only store encrypted password vaults without knowledge of the actual contents.

In general, users are required to create a master password to authenticate and manage password vaults. We characterize the interaction between the client and the PM’s server as the *Master-Password-Protected Password Management (M3PM)* protocol. We define M3PM as a cryptographic protocol that authenticates the client and ensures the password vault remains oblivious to the server side. Secure encryption of password vaults does not necessarily ensure a secure M3PM protocol. An inappropriate combination of authentication and encryption mechanisms can introduce vulnerabilities (see Sec. 5.4), and M3PM protocols encompass broader security properties beyond encryption (see Sec. 4.1). To the best of our knowledge, this is the first study to

systematically identify and analyze the security of M3PM protocols. We focus on three research questions (RQs):

- **RQ1:** What M3PM protocols have been adopted by existing password managers?
- **RQ2:** What defines a secure M3PM protocol?
- **RQ3:** Are existing password managers using M3PM protocols secure under our security definitions?

To address these three questions, we develop a systematic methodology to understand the status quo of M3PM protocols. This involves collecting and refining information about M3PM protocols from 43 major PMs sourced from academia and industry. We summarize the security properties of M3PM protocols and formalize ideal functionalities to describe what constitutes an ideally secure M3PM protocol within the universal composability (UC) framework [18]. This enables a systematic analysis of the M3PM protocols to identify security vulnerabilities. In summary, our contributions include:

- **A methodology for identifying M3PM protocols.** We introduce a systematic methodology for identifying M3PM protocols across 36 popular industrial PMs and seven representative academic PMs. Information about these protocols is sourced from three primary channels: documentation, data logs, and application code. We use an iterative process of collection and verification to accurately identify information on M3PM protocols, cryptographic algorithms, and metadata encryption.
- **Ideal functionalities of M3PM protocols.** We formalize the security properties of M3PM protocols and present an ideal functionality $\mathcal{F}_{M3PM}$ within the UC framework. During our analysis of the collected M3PM protocols, we iteratively revise and extend this functionality to distinguish between explicit authentication $\mathcal{F}_{M3PM-EA}$ and implicit authentication $\mathcal{F}_{M3PM-IA}$. Inspired by previous work on password-related protocols [7], [19], [22], [25], [28], we introduce a relaxed functionality $\mathcal{F}_{rM3PM}$ to address guessing attacks and present $\mathcal{F}_{adM3PM}$ to accommodate protocols with additional secrets.
- **Security analysis for M3PM protocols.** We categorize four types of offline master password guessing attacks, and find 38 of 43 PMs cannot resist at least one type of offline guessing attack. We summarize the circumstances that each M3PM protocol fails to resist offline guessing attacks and conclude that the current design of M3PM protocols, without additional secrets or devices, suffers from the low entropy of master passwords and heavily relies on the effectiveness of slow hash or memory-hard functions. Additionally, we find fully plaintext communication in Delinea Web, and uncover an oracle attack where a corrupted server can learn the encryption key of Passbolt. Overall, we give provable analysis of eight distinct M3PM protocols and prove the M3PM protocols of 1Password and Multipass realize $\mathcal{F}_{adM3PM-EA}$.

Responsible disclosure and ethical considerations. Given the potential risks of users leaking their passwords, we have disclosed our findings to the two affected PM vendors: Delinea Web and Passbolt. In particular, we contacted Delinea Web about the vulnerability. They acknowledged receipt of our email, but they have not provided a detailed response at the time of paper preparation. Additionally, we engaged in an in-depth discussion with Passbolt regarding the identified vulnerability and provided recommendations to mitigate the risks (see details in Sec. 5.4).

2. Preliminary

2.1. Password Managers

Password managers (PMs) are tools that help users manage their accounts. The basic management function of PMs is to memorize passwords so that users can effortlessly delegate their password management responsibilities to the PMs, retrieving passwords as needed. Modern PMs offer the capability to generate random passwords, and users can tailor password settings, including length and character composition. During the login process on respective websites, the autofill feature in PMs automatically populates users' credentials, alleviating the need for manual typing. This enhanced convenience attributed to autofill is a major driver behind the adoption of PMs [44]. Furthermore, PMs employ tools like the password strength meter, such as Zxcvbn [58], to assess and identify weak passwords. Additionally, PMs utilize compromised credential checking services, such as Have I Been Pwned [5], to identify and alert users about any passwords that may have been compromised or leaked.

In this work, we focus on the basic management function, i.e., memorizing passwords. Depending on the way of memorizing passwords, PMs can be categorized into two types [37]: (1) Retrieval PMs (e.g., 1Password and LastPass) store users' credentials in digital memory, commonly referred to as *password vaults*. These credentials are then retrieved as needed during the user authentication process; (2) Generative PMs (e.g., LessPass and SPHINX [48]) create users' credentials on demand by inputting the master password and additional information such as URLs and usernames into deterministic functions, such as OPRF [48] or hash functions [47]. The result is the creation of distinct passwords for different user accounts. In essence, retrieval PMs store encrypted user-chosen credentials, like an encrypted database, whereas generative PMs generate passwords on behalf of different accounts.

2.2. Related Work

At ESORICS'12, Gasti and Rasmussen [24] investigated the storage format of password vaults. They gave two security definitions for password vaults: (1) Indistinguishability of databases game corresponding to passive attackers; (2) Malleability of chosen database game corresponding to active attackers. Their analysis reveals that unencrypted metadata would leak users' privacy and give attackers the advantage of distinguishing users' password vaults and tampering vaults without integrity checks. Their work is the first

rigorous analysis for PMs, but it analyzed the storage format instead of the M3PM protocol, which neglects authentication of retrieval password vaults and offline guessing attacks of the master passwords. Furthermore, their work analyzed 13 PMs in industry, while we give a more systematic analysis across 36 industrial PMs and seven academic PMs. In 2020, Oesch and Ruoti [41] replicated Gasti and Rasmussen’s [24] study, finding PMs have improved in protecting metadata but still leave some plaintext vulnerable. Replicating their analysis, we find that after three years, 30 out of 43 PMs still leave plaintext metadata (see details in Appendix E of the full paper at <https://bit.ly/3zBPVUO>).

Zhao et al. [63] analyzed the storage of two cloud-based PMs and found that insecure storage methods (e.g., the insecure relationship between master passwords and authenticators) can leave the password vaults vulnerable. In the same year, Zhao and Yue [62] showed five browser-based PMs would be cracked once the encrypted information is stolen. These works pointed out the potential vulnerability of offline guessing attacks on master passwords and gave us insights to conduct a systematic and rigorous analysis.

Bojinov et al. [14] and Chatterjee et al. [20] highlighted that encryption using the master password alone cannot withstand offline guessing attacks in the event of a leakage of the encrypted password vault. They proposed novel schemes, called honey vaults, to resist offline guessing attacks on leaked vaults. Inspired by their work, we formalize the process of offline guessing attacks and emphasize that the adversary’s prior knowledge significantly influences the ability to attack honey vaults (see details in Sec. 5.1).

At CRYPTO’23, Davies et al. [22] formally analyzed WhatsApp’s backup protocol and proposed a password-protected key retrieval (PPKR) functionality in the UC framework. Our M3PM functionalities share similarities with the PPKR functionality, but there are three main differences: (1) The PPKR functionality employs one password to retrieve one key, whereas the M3PM functionality utilizes one master password *mpw* to retrieve multiple passwords. This distinction is crucial as a secure M3PM protocol must safeguard one password even if other passwords are leaked. (2) The M3PM functionality distinguishes explicit and implicit authentication. (3) The PPKR functionality allows an adversary to attempt 10 offline guesses, while our relaxed M3PM functionality does not restrict the guessing numbers, which is attributed to the methods in Sec. 6.1.

3. M3PM Protocols Collection

To address **RQ1**: *What M3PM protocols have been adopted by existing password managers?* we identify the diverse M3PM protocols in use. In this section, we elaborate on the process by which we select 43 PMs and collect information regarding the M3PM protocol for each PM.

3.1. PMs Selection

To comprehensively analyze various password management schemes, we exhaustively identify PMs from academic and industrial sources in September and October 2023.

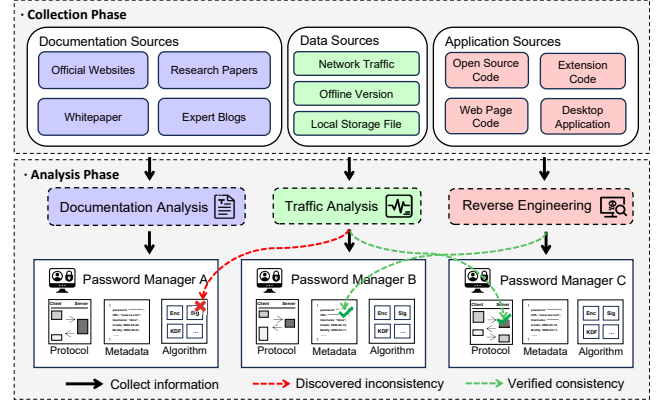


Figure 1: An overview of our methodology. We collect information from documentation, data logs, and application codes. Documentation analysis gathers initial data, while traffic analysis and reverse engineering are used to collect additional information and verify the consistency.

For industrial PMs, we first collect PMs that are popular with users. We create core search terms (i.e., “password manager”, “password vault”, “password management”, and “password store”) and enter each core term as a query into the search box on the Chrome Web Store, Google Play Store, Firefox Add-ons, Edge Add-ons, and Apple’s App Store. We keep the top 30 PMs displayed in each query and remove duplicated PMs. To avoid overlooking popular PMs, we include those recommended annually by sites like passwordmanager.com, U.S. News, PCMag, Cybernews, CNET, Tom’s Guide, Forbes, and Security.org.

Leveraging the aforementioned core search terms, we utilize Github to select relevant open-source PMs and retain the PMs with over one thousand stars. We consider PMs with over 1,000 stars on GitHub, as this metric indicates a stronger appeal and approval among developers. All these above steps result in 145 PMs from the industry.

For academic PMs, we use the core terms to search in DBLP and Google Scholar, and we only collect new M3PM protocols. Finally, we collect 15 PM schemes published in IEEE S&P, USENIX Security, WWW, UbiComp, AsiaCCS, ESORICS, ACSAC, ICDSC, NSPW, Computers&Security, SOUPS, IEEE TDSC, JCST, and SecureComm.

We exclude PMs that are fewer than 10,000 installations, academic PMs with a primary focus other than security, and PMs that cannot be set up or launched. This process results in a final selection of 43 PMs for our in-depth research, and we summarize these 43 PMs in Table 1.

3.2. Collection Methodology

In Fig. 1, we present an overview of our collection methodology. We gather information from various sources, including documentation sources, data sources, and application sources. All this information is publicly available, and we do not employ it for any exploitation of PMs.

Documentation sources. For documentation sources, we use the names of corresponding PMs as core search terms in popular search engines, Google and Bing. According to

the results of search engines, we collect every descriptive documentation of PMs, including industrial whitepapers, README files of open-source PMs, expert blogs, and other support documentation on PMs' official websites.

We first search for the descriptions about security and encryption on each documentation. Subsequently, we extract the relevant information about algorithms and protocols from the collected documentation. For example, Bitwarden has detailed information about its security architecture. This includes specifics on how Bitwarden authenticates users and the encryption methods employed for storing users' credentials while excluding details on other security aspects like password sharing and single sign-on.

For PMs without sufficient public information, we collect the limited available information and formulate sensible and conservative predictions. For example, Norton PM's official website only reveals that they use AES-256 as an encryption method and encrypt the password vaults; then, we record this encryption method and determine that Norton is a retrieval PM. Our goal is not an exhaustive collection of all online documentation but rather acquiring a sufficient amount for further in-depth research.

Data sources. For data sources, we use the web page version of PMs and install browser extensions on Edge and Chrome. For each PM, we first create an account (we delete these accounts after research to avoid unnecessary burdens for PM servers). Furthermore, we use Burp Suite to record the HTTPS traffic. Following the account creation, we proceed with a series of essential operations, including logging in, opening the vault, adding credentials, changing credentials, deleting credentials, and logging out. In the subsequent content, we refer to these operations collectively as *PM operations*, considering them fundamental actions for collecting information on M3PM protocols.

Our collection of plaintext network traffic without TLS aims to discern the communication patterns associated with M3PM protocols. We do not emphasize TLS because in the context of M3PM protocols, the server is assumed to be malicious, rendering the TLS mechanism between a client and a malicious server ineffective (see details in Sec. 4.1). Besides, we record crucial data fields for further code analysis. For example, we find a data field called "encrypted_username" from network traffic with LastPass, and then we search for this string in the source code to find where and how the algorithm encrypts usernames.

Furthermore, we install the desktop version of PMs on different platforms, such as Windows, Linux, and MacOS, and search for local storage files. We record the plaintext in data files, and we also record crucial data fields for further code analysis. Specifically, Zoho Vault provides an offline version, so we collect the offline version for further research.

Application sources. For application sources, we conduct static and dynamic analyses of applications. We gather code from open source code, extension code, and web page code on the client side. We use the information collected from documentation and data sources to form core terms (e.g., "AES256", "PBKDF2"), and locate the cryptographic algorithms in the source code.

For dynamic analysis, we use browser developer tools to set breakpoints at the locations identified by static analysis. We then execute the *PM operations*, recording the precise algorithms associated with each operation. During dynamic analysis, we find the exact parameter values for cryptographic algorithms, such as iterations for key derivation functions. To address the challenges posed by JavaScript obfuscation and diverse front-end technology stacks, we conduct iterative dynamic analyses and validate the results collaboratively among the authors.

3.3. Collection Results

In this section, we answer **RQ1** and give an overview of our collection results, including M3PM protocols, cryptographic algorithms in protocols, and metadata encryption.

All the results are made open at <https://bit.ly/3zBPVUO>, and we record as many details as we can to enhance the reproducibility. Overall, (1) 24 of 36 industrial PMs are analyzed using reverse engineering; (2) 32 PMs provide useful information in local data files or network data packets; (3) Only six PMs are primarily analyzed using documentation. In all, 27 PMs collect sufficient information for in-depth analysis. Note that when the data logs provide sufficient information, we do not perform reverse engineering. For instance, during our network capture, we observed that Delinea Web does not encrypt any data, which eliminates the need for further static or dynamic analysis.

M3PM protocols. Throughout our collection process, we identified that M3PM protocols can be categorized into two distinct stages: authentication and password processing. The authentication stage primarily facilitates explicit authentication for client-to-server communication. M3PM protocols adopt several approaches for explicit authentication: (1) Sending the hash of master passwords (*Hashed mpw*); (2) Using master passwords to decrypt protected authentication tokens and sending the tokens to authenticate (*Protected tokens*); (3) Using password-authenticated key exchange (PAKE) protocols like *SRP*; (4) Using master passwords to answer the server's challenge (*mpw challenge*); (5) Using public keys to answer the challenge from the server (*pk challenge*). We summarize the authentication phase of the protocol in Fig. 2, omitting the registration initialization.

After successful authentication, the client and server initiate password processing. Methods include encrypting/decrypting passwords with a key derived from the master password (*mpw encrypt*) or with a key generated by devices (*device-key encrypt*), both of which involve explicit authentication. Implicit authentication methods include encoding passwords before encryption (*encode-then-encrypt*) and using metadata and the master password to generate passwords (*generative*). The password-processing phase is depicted in Fig. 3, omitting the vault registration for brevity.

Cryptographic algorithms. We summarize the cryptographic algorithms in M3PM protocols. These cryptographic algorithms encompass symmetric encryption, asymmetric encryption, hash functions, and key derivation functions. Within M3PM protocols, these cryptographic algorithms are

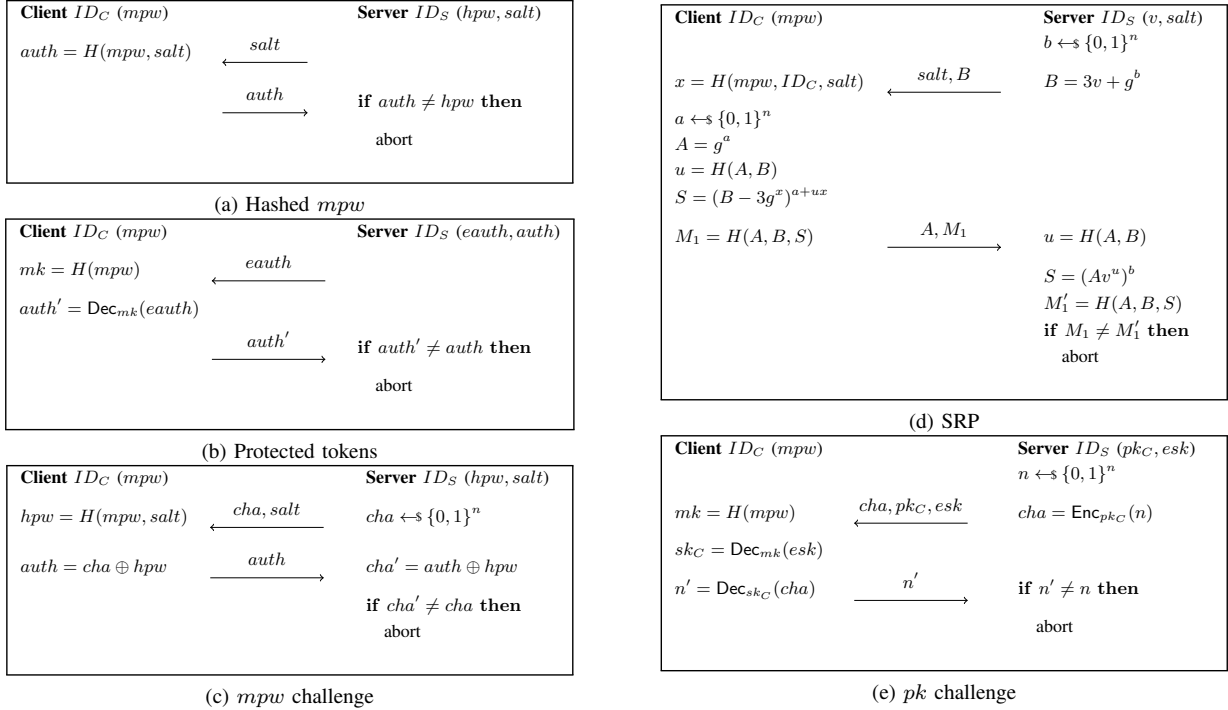


Figure 2: The authentication stage of M3PM protocols. During registration, the client sends authenticators to the server through a trusted channel. For *Hashed mpw*, the server receives the hash of the master password $hpw = H(mpww, salt)$ and the salt; For *Protected tokens*, the server receives the encrypted and plaintext authenticators $eauth = \text{Enc}_{H(mpww)}(auth)$ and $auth$; For *mpw challenge*, the server receives the hash of the master password $hpw = H(mpww, salt)$ and the salt; For *SRP*, the server receives the authenticator derived from the master password $v = g^{H(mpww, ID_C, salt)}$ and the salt; For the *pk* challenge, the server receives the public key pk_C and the encrypted private key $esk = \text{Enc}_{H(mpww)}(sk_C)$.

presented as abstract algorithms, accompanied by specific security assumptions (see details in Appendix A). Details about the choices of cryptographic algorithms and parameters are in Appendix D at <https://bit.ly/3zBPVUO>.

Metadata privacy. The encryption status of metadata not only impacts user privacy but also has implications for distinguishability and malleability [24], [41]. In our work, we divide the data in password vaults into five different types: secret, login metadata, item state metadata, password hygienist metadata, and users' profiles. See details and discussions in Appendix E at <https://bit.ly/3zBPVUO>.

4. Security Definitions of M3PM Protocols

To address **RQ2: What defines a secure M3PM protocol?** we design a set of security definitions for the M3PM protocol and formalize these security goals within the UC framework [18]. The UC framework ensures that the security of a protocol is preserved even when it is executed alongside other protocols. We provide an informal description about our security goals and an overview of our formal security functionalities. These functionalities are illustrated in Fig. 4, with detailed descriptions provided in Appendix B.

4.1. Security Goals

Generally, an adversary should not gain any knowledge of the master password mpw or the passwords pwd stored in

password vaults. Given recent data breaches affecting PMs, such as the LastPass breach [8], we model the adversary as having the capability to compromise the server and access all stored data. Additionally, many PMs, including Bitwarden, assert that they have “zero knowledge” of users' passwords. Therefore, we assume that a malicious server should not learn any information about the passwords pwd . The malicious server can send messages that deviate from the original M3PM protocol. We assume that the communication channel is secured using TLS and that neither the client nor the server operates in a trusted environment.

Furthermore, we summarize below the expected security properties we have formalized.

- **Indistinguishability of passwords** [24], [41]: The password initialized by an honest client remains unknown to the malicious server. In other words, any different passwords initialized by the client are indistinguishable to the malicious server.
- **Non-malleability of passwords** [24], [41]: The malicious server cannot tamper with the initialized password without the client's explicit notice.
- **Oblivious of master password** [14], [20], [32], [48], [62]: The initialization and retrieval phases do not leak any information about the master password to the malicious server, ensuring its confidentiality.
- **Pseudorandom generated passwords** [41]: The

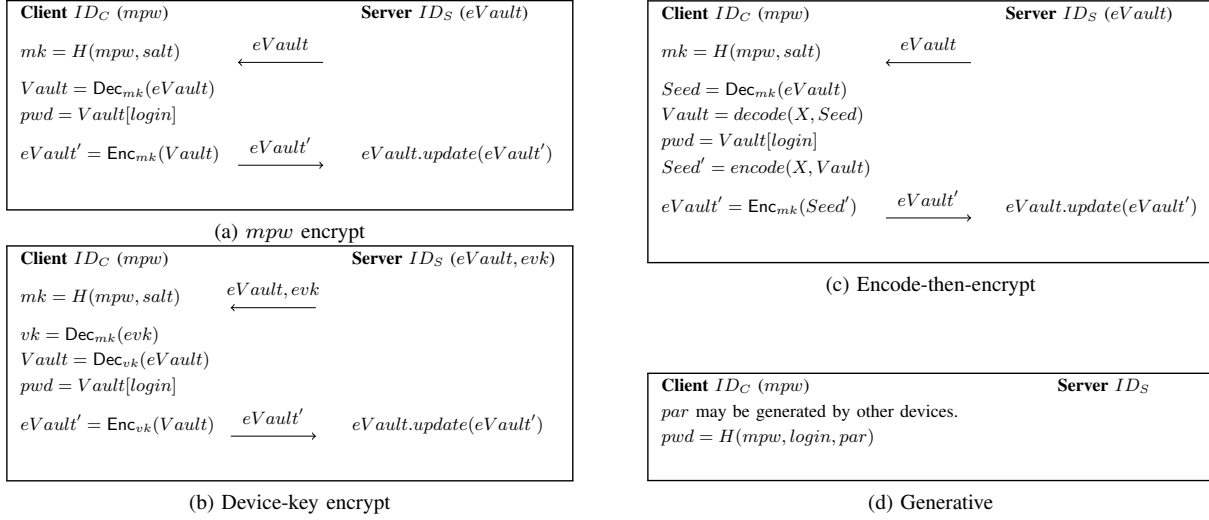


Figure 3: The password processing stage of M3PM protocols. During vault initialization, the client initializes the encrypted vault and sends it to the server. For *mpw encrypt*, the server receives the encrypted vault $eVault = \text{Enc}_{H(mpw, salt)}(Vault)$; For *Device-key encrypt*, the server receives another encrypted key $evk = \text{Enc}_{H(mpw, salt)}(vk)$; For *Encode-then-encrypt*, the server receives $eVault = \text{Enc}_{H(mpw, salt)}(\text{encode}(X, Vault))$; For *Generative*, the server receives parameters *par*.

password generated in the initialization phase is pseudorandom and completely independent of any other previously initialized passwords, even if the server acts maliciously.

- **Resistance to password guessing attack** [14], [20], [48]: The adversary cannot conduct guessing attack on master password. This security property is relatively strong to M3PM protocols, as users rely solely on the master password to protect their vaults (discussed further in Sec. 5.1). Therefore, in the functionality $\mathcal{F}_{M3PM}$, we include a TESTMPW query to model the malicious server’s ability to conduct guessing attacks on the master password. This functionality is practical because there are several effective methods beyond protocol design to mitigate guessing attacks (discussed in Sec. 6.1).
- **Resistance to the leakage of partial passwords** [48]: In our functionalities, the security ensures that the leakage of partial passwords does not compromise the security of other passwords in the vault. This security property is captured by modeling the initialization and retrieval processes for one credential at a time, as well as by ensuring the independence of each password.
- **Authentication** [22]: When retrieving passwords, the client must be authenticated by using the same master password used in the initialization phase.

Remark 1. Previous work [24], [41] highlights the importance of indistinguishability and non-malleability of metadata. In our functionalities, we can extend protection to metadata by modifying *pwd* to a tuple $(pwd, metadata)$. Our review shows that 30 of 43 PMs do not encrypt all metadata (see details in Appendix E of the full paper), which could limit our analysis of innovative M3PM protocols. For instance, stopping security analysis due to unencrypted

metadata in 1Password would prevent us from recognizing the security of their dual-key mechanisms (see Fig. 5). As unencrypted metadata is a straightforward issue to mitigate, we present the functionalities without metadata protection.

4.2. Overview of Functionalities

To cover the various M3PM protocols, we develop a set of functionalities and iteratively revise them during our analysis of M3PM protocols. Firstly, we present $\mathcal{F}_{M3PM}$ to cover the basic security properties of initializing and retrieving a credential. We then identify two types of authentication: explicit and implicit, each requiring different functionalities to capture their flow. For explicit authentication, when the client fails to authenticate, the server responds with a FAIL signal. For implicit authentication, the server responds with a random and independently incorrect password to the client upon authentication failure. Therefore, we extend $\mathcal{F}_{M3PM}$ to $\mathcal{F}_{M3PM-EA}$ and $\mathcal{F}_{M3PM-IA}$. Note that authentication failures in $\mathcal{F}_{M3PM-IA}$ will result in the generation of a password that follows the Zipf’s law distribution, as honey encryption will generate a decoy password that also follows the Zipf’s law distribution upon authentication failure.

Our analysis shows that 34 of the 43 M3PM protocols (see Table 1) cannot fulfill $\mathcal{F}_{M3PM-EA}$ or $\mathcal{F}_{M3PM-IA}$, because they fail to resist offline guessing attacks if the server is compromised. However, we find that 35 PMs use PBKDF2 or Argon2 [10] to mitigate offline guessing attacks (see details in Appendix D of the full paper). These functions can reduce the success rate of offline guessing master passwords. Therefore, we introduce a relaxed functionality $\mathcal{F}_{rM3PM}$ containing $\mathcal{F}_{rM3PM-EA}$ and $\mathcal{F}_{rM3PM-IA}$ in Fig. 4. Notably, we do not include a query for online guessing for simplicity. This does not compromise correctness because an M3PM protocol with $\mathcal{F}_{rM3PM}$ cannot resist both online and offline

Initialization Phase

On input $(\text{INITC}, \text{sid}, ID_C, ID_S, mpw, \text{login}, \text{pwd})$ from ID_C (or $\mathcal{A}$ if ID_C is corrupted):

- IC.1 If $\text{pwd} = \epsilon$ then: $\text{pwd} \leftarrow \{0, 1\}^n$, and mark it GENERATED //Generate a random password for a user.
- IC.2 Record $\langle \text{INITC}, \text{sid}, ID_C, ID_S, mpw, \text{login}, \text{pwd} \rangle$, overwriting existing record $\langle \text{INITC}, \text{sid}, ID_C, ID_S, *, *, * \rangle$
- IC.3 Send $(\text{INITC}, \text{sid}, ID_C, ID_S)$ to $\mathcal{A}$ and ID_S

On input $(\text{INITS}, \text{sid}, ID_C, ID_S)$ from ID_S (or $\mathcal{A}$ if ID_S is corrupted):

- IS.1 Retrieve record $\langle \text{INITC}, \text{sid}, ID_C, ID_S, [mpw], [\text{login}], [\text{pwd}] \rangle$ //The client has already started initialization.
- IS.2 Record $\langle \text{VAULT}, ID_C, ID_S, mpw, \text{login}, \text{pwd} \rangle$ and send $(\text{INITS}, \text{sid}, ID_C, ID_S)$ to $\mathcal{A}$

On input $(\text{COMPLETEINITC}, \text{sid}, ID_C, ID_S)$ from $\mathcal{A}$: //The client outputs a password and completes the protocol.

CIC.1 Retrieve record $\langle \text{INITC}, \text{sid}, ID_C, ID_S, mpw, *, [\text{pwd}] \rangle$ and delete it

CIC.2 If there is a record $\langle \text{VAULT}, ID_C, ID_S, mpw', *, * \rangle$ marked STORED, then:

- a) If $mpw = mpw'$, send success to ID_C and $\mathcal{A}$
- b) If $mpw \neq mpw'$, send failure to $\mathcal{A}$ and ID_C . Delete record $\langle \text{INITC}, \text{sid}, ID_C, ID_S, mpw, *, [\text{pwd}] \rangle$

On input $(\text{COMPLETEINITS}, \text{sid}, ID_C, ID_S)$ from $\mathcal{A}$: //The server stores a record and completes the protocol.

CIS.1 Retrieve record $\langle \text{VAULT}, ID_C, ID_S, [mpw], *, * \rangle$ not marked STORED

CIS.2 If there is a record $\langle \text{VAULT}, ID_C, ID_S, mpw', *, * \rangle$ marked STORED, then:

- a) If $mpw = mpw'$, mark $\langle \text{VAULT}, ID_C, ID_S, mpw, \text{login}, \text{pwd} \rangle$ STORED and send success to ID_S and $\mathcal{A}$
- b) If $mpw \neq mpw'$, send failure to $\mathcal{A}$ and ID_S , and delete $\langle \text{VAULT}, ID_C, ID_S, mpw, *, * \rangle$

Retrieval Phase

On input $(\text{RETRC}, \text{sid}, ID_C, ID_S, mpw', \text{login})$ from ID_C (or $\mathcal{A}$ if ID_C is corrupted):

- RC.1 Record $\langle \text{RETRC}, \text{sid}, ID_C, ID_S, mpw', \text{login} \rangle$, overwriting any record $\langle \text{RETRC}, \text{sid}, ID_C, ID_S, *, \text{login} \rangle$
- RC.2 Send $(\text{RETRC}, \text{sid}, ID_C, ID_S)$ to $\mathcal{A}$ and ID_S

On input $(\text{RETRS}, \text{sid}, ID_C, ID_S)$ from ID_S (or $\mathcal{A}$ if ID_S is corrupted):

- RS.1 Retrieve record $\langle \text{RETRC}, \text{sid}, ID_C, ID_S, [mpw'], [\text{login}] \rangle$
- RS.2 If no record $\langle \text{VAULT}, ID_C, ID_S, [mpw], *, * \rangle$ is marked STORED, send $(\text{RETRRESULT}, \text{sid}, \text{FAIL})$ to ID_S
- RS.3 Append mpw to $\langle \text{RETRC}, \text{sid}, ID_C, ID_S, mpw', \text{login} \rangle$, overwriting any $\langle \text{RETRC}, \text{sid}, ID_C, ID_S, *, *, * \rangle$
- RS.4 Send $(\text{RETRS}, \text{sid}, ID_C, ID_S, mpw \stackrel{?}{=} mpw')$ to $\mathcal{A}$

On input $(\text{COMPLETERETRC}, \text{sid}, ID_C, ID_S)$ from $\mathcal{A}$: //The client outputs a password and completes the protocol.

CRC.1 Retrieve $\langle \text{RETRC}, \text{sid}, ID_C, ID_S, [mpw'], [\text{login}], [mpw] \rangle$. Mark it RETRIEVED. If it is RETRIEVED, delete it.

CRC.2 If $mpw = mpw'$ then retrieve the record $\langle \text{VAULT}, ID_C, ID_S, mpw', \text{login}, [\text{pwd}] \rangle$ and set $\text{pwd}' \leftarrow \text{pwd}$ and send $(\text{RETRRESULT}, \text{sid}, \text{SUCCESS})$ to $\mathcal{A}$ if ID_C is not corrupted

CRC.3 If $mpw \neq mpw'$, determine the output with two options:

- a) If pwd is marked GENERATED then set $\text{pwd}' \leftarrow \{0, 1\}^n$. Otherwise, set $\text{pwd}' \leftarrow \text{Zipf}(n)$
- b) Set $\text{pwd}' \leftarrow \text{FAIL}$, and send $(\text{RETRRESULT}, \text{sid}, \text{FAIL})$ to $\mathcal{A}$ if ID_C is not corrupted

CRC.4 Send $(\text{RETRRESULT}, \text{sid}, \text{pwd}')$ to ID_C

On input $(\text{COMPLETERETRS}, \text{sid}, ID_C, ID_S)$ from $\mathcal{A}$: //The server retrieves the record and completes the protocol.

CRS.1 Retrieve $\langle \text{RETRC}, \text{sid}, ID_C, ID_S, [mpw'], [\text{login}], [mpw] \rangle$. Mark it RETRIEVED. If it is RETRIEVED, delete it.

CRS.2 If $mpw = mpw'$, send $(\text{RETRRESULT}, \text{sid}, ID_C, ID_S, \text{SUCC})$ to ID_S (or $\mathcal{A}$ if ID_S is corrupted)

CRS.3 If $mpw \neq mpw'$, send $(\text{RETRRESULT}, \text{sid}, ID_C, ID_S, \text{FAIL})$ to ID_S (or $\mathcal{A}$ if ID_S is corrupted)

On input $(\text{TESTMPW}, ID_C, ID_S, mpw)$ from $\mathcal{A}$ if server ID_S is corrupted:

TM.1 If there exists a $\langle \text{VAULT}, ID_C, ID_S, mpw, *, * \rangle$, then send CORRECT to $\mathcal{A}$. Otherwise, send WRONG to $\mathcal{A}$.

Figure 4: Ideal functionality $\mathcal{F}_{M3PM}$ for master-password-protected password management protocols. The boxed code describes the explicit authentication $\mathcal{F}_{M3PM\text{-}EA}$, and we can drop it to have the implicit authentication $\mathcal{F}_{M3PM\text{-}IA}$. The dashed box is for relaxed functionality $\mathcal{F}_{rM3PM}$. The $\mathcal{F}_{adM3PM}$ needs change all mpw into a tuple $(mpw, \text{SecretK})$.

guessing attacks simultaneously. A single query TESTMPW can capture both types of attacks during the proof.

Finally, we find that 1Password and Multipassword adopt additional secret values and achieve strong security against all kinds of offline guessing attacks (see Table 1). Therefore, we introduce another functionality $\mathcal{F}_{adM3PM}$ which captures additional secrets enhanced M3PM protocols.

Remark 2. In our functionalities, we only consider two parties: a client and a server. In fact, there are also M3PM protocols containing one or three parties. For one party, we designate the local PM as the entity through which users retrieve password vaults on a specific device without involving network traffic. Strictly speaking, this scenario may not qualify as a M3PM protocol, but we can analyze this scenario by assigning with a server to facilitate synchronization. For the three-parties protocol, there is another assistant device. In our functionalities, we only consider two participants for two reasons: two-party protocols are beneficial for UC framework [18]; two-party protocols can be extended to three within the UC framework, and one-party protocols can be modeled as a secure functionality.

5. Security Analysis

To answer **RQ3: Are existing password managers secure under our security definitions?** we give a security analysis of all M3PM protocols. The entire analysis process takes nearly four months. During our iterative analysis, any proof failure would lead to the identification of potential attacks.

Sec. 5.1 outlines that the authentication schemes in Fig. 2 and the password processing schemes in Fig. 3 are vulnerable to offline guessing attacks. In our analysis, we show that only 1Password and Multipassword achieve $\mathcal{F}_{adM3PM-EA}$ which is owed to the entropy of the secret key. Other protocols at most achieve $\mathcal{F}_{rM3PM-EA}$ or $\mathcal{F}_{rM3PM-IA}$, because they all suffer offline guessing attacks. In total, there are eight distinct protocols. Sec. 5.3 presents the security proof for 1Password's protocol and explains how this proof can be extended to other M3PM protocols with the same combination: explicit authentication with *mpw encryption* or *device-key encryption* scheme. For the *generative* scheme, we leave the proof of SPHINX [48] in Appendix G of the full paper at <https://bit.ly/3zBPVUO>.

Furthermore, Delinea Web PM does not adopt any cryptographic method on the client side. Passbolt faces an oracle attack which we describe in Sec. 5.4. Six PMs without a provable security result cannot be proven secure in our model: Norton PM, Chrome PM, and Passworden do not have enough information for protocol analysis; Amnesia [57], BluePass [32], and Tapas [38] have three parties in protocols. Detailed results are summarized in Table 1.

5.1. Offline Guessing Attacks

The underlying vulnerability of offline guessing attacks comes from two facts: low entropy and offline verification. The low entropy of passwords brings unavoidable brute force guessing, and Zipf-distribution compresses the

guessing space of passwords [53]. A password commonly has 2^{20-22} bit entropy [15], while AES has three options of key length, i.e., 128, 192, and 256 bits. Consequently, successfully guessing a password carries a non-negligible probability compared to guessing an ordinary random key.

Another reason causing offline guessing attacks is that adversaries find a way to verify the success of password guessing offline. For example, the adversary may exploit the database of the server and then obtain all password authenticators $h_{pw} = H(mpw, salt)$. Then the adversary can guess passwords mpw' and compare the hash results $h_{pw'} = H(mpw', salt)$ with the authenticators to verify the correctness of the password guesses.

Formally, we formalize this verification process as an algorithm *GuessVerify*. The *GuessVerify* takes three inputs: a string m which is the message for verification, a distribution X which gives the probabilities of the m to be a correct guess, and a threshold σ which represents the confidence for the adversary about the guess. If the probability of m in X is larger than σ then the algorithm returns $\delta = 1$ representing that the verification is correct. For example, the adversary knows passwords follow Zipf-distribution [53], then she sets the distribution X to be Zipf-distribution, and she believes that the probability more than 0.1 represents a correct guess, then she sets the threshold σ to be 0.1. Therefore, when the adversary decrypts the vault, a value `abc12345` has a high probability of being a password in Zipf-distribution, while `Q!= 'GhEuk76!WH+92Rf` has a low probability of being a password even it is generated by a password generator. We use distribution X to give a general description for the process of verification, and for a specific situation, such as the hashed password, the distribution can be $\Pr[h_{pw} = H(mpw', salt)] \approx 1$ and $\Pr[h_{pw} \neq H(mpw', salt)] \approx 0$.

These explicit authentication schemes inevitably suffer from offline guessing attacks because the server needs to store an authenticator for the master password. For *hashed mpw*, the corrupted server ID_S has $h_{pw} = H(mpw, salt)$ and the *salt*, then ID_S can guess a master password mpw' and calculates $h_{pw'} = H(mpw', salt)$. The ID_S can validate the correctness of guesses by checking $h_{pw} \stackrel{?}{=} h_{pw'}$. For *protected tokens*, the ID_S has $e_{auth} = \text{Enc}_{H(mpw)}(auth)$ and *auth*, then ID_S can guess the master password mpw' and decrypt the *auth* by $auth' = \text{Dec}_{H(mpw')}(e_{auth})$, then ID_S can verify the guess by checking $auth \stackrel{?}{=} auth'$. For *SRP*, the corrupted server ID_S has $v = g^{H(mpw, ID_C, salt)}$ and *salt*, then ID_S can guess the master password mpw' , calculate $v' = g^{H(mpw', ID_C, salt)}$, and verify $v \stackrel{?}{=} v'$. For *mpw challenge*, the server has the plaintext of the master password, and for *pk challenge* the server ID_S guesses the master password and decrypts the private key $sk'_C = \text{Dec}_{H(mpw')}(esk)$, and uses the public key pk_C to validate the correctness of the guess.

In the password-processing stage, the situation is more complicated. For *mpw encrypt*, the server ID_S has the encrypted password vault $eVault$. Then ID_S guesses a master password mpw' and calculates the key $mk' =$

TABLE 1: Overview of security in password managers. [†]

Password manager@Version	Protocol Syntax		Resist offline guessing*				Metadata privacy	Enough details	Provable Security
	Authentication	Password processing	I	II	III	IV			
Bitwarden@2023.9.1	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
1Password@8.10.16	SRP+two-key [‡]	device-key encrypt	●	●	●	●	○	●	$\mathcal{F}_{adM3PM-EA}$
LastPass@4.123.0	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
Dashlane@6.2342.0	protected tokens	<i>mpw</i> encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
RoboForm@9.5.2	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
Keeper@16.10.9	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
NordPass@5.8.21	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
Zoho Vault@3.8	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
KeePass@2.55	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	●	●	$\mathcal{F}_{rM3PM-EA}$
KeePassXC@2.7.6	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	●	●	$\mathcal{F}_{rM3PM-EA}$
Dropbox PM@3.26.0	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
DualSafe PM@1.4.21	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	●	●	$\mathcal{F}_{rM3PM-EA}$
Enpass@6.9.1	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	○	○	$\mathcal{F}_{rM3PM-EA}$
Norton PM@8.1.0.73	<i>mpw</i> challenge	<i>mpw</i> encrypt	●	○	●	○	●	○	-
Avira PM@2.19.13.44521	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
McAfee True Key@4.3.1.9339	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
Sticky PM@8.8.3.1629	protected tokens	<i>mpw</i> encrypt	●	○	●	○	○	○	$\mathcal{F}_{rM3PM-EA}$
Password Boss@5.5.5138.0	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	○	○	$\mathcal{F}_{rM3PM-EA}$
Multipass@0.93.36	SRP+two-key [‡]	device-key encrypt	●	●	●	●	○	●	$\mathcal{F}_{adM3PM-EA}$
Passworden@1.3.3	protected tokens	<i>mpw</i> encrypt	●	○	●	○	○	○	-
Delinea Web PM@3.7	-	-	●	○	●	○	○	●	✗
LogMeOnce@7.8.0	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
Passbolt@4.4.2	<i>pk</i> challenge	<i>mpw</i> encrypt	●	○	●	○	○	●	✗
IronVest@9.8.15	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
SafeInCloud PM@22.3.4	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	○	○	$\mathcal{F}_{rM3PM-EA}$
Password Safe@3.64.1	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	●	●	$\mathcal{F}_{rM3PM-EA}$
Bitdefender PM@3.21.1247	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	○	○	$\mathcal{F}_{rM3PM-EA}$
iCloud Keychain@Ventura 13.5.2	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	●	○	$\mathcal{F}_{rM3PM-EA}$
Chrome PM@119.0.6045.160	-	device-key encrypt	●	○	●	○	○	○	-
KeeWeb@1.18.7	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	●	●	$\mathcal{F}_{rM3PM-EA}$
Lesspass@9.1.9	-	generative	●	●	○	○	●	●	$\mathcal{F}_{rM3PM-IA}$
Gopass@1.15.10	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
Padloc@4.3.0	SRP	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
Authpass@1.9.9	hashed <i>mpw</i>	<i>mpw</i> encrypt	●	○	●	○	●	●	$\mathcal{F}_{rM3PM-EA}$
Gokey@0.1.2	-	generative	●	●	○	○	●	●	$\mathcal{F}_{rM3PM-IA}$
TeamPass@3.0.10	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	○	$\mathcal{F}_{rM3PM-EA}$
SPHINX [48]	-	generative	●	●	●	○	●	●	$\mathcal{F}_{rM3PM-IA}$
BluePass [32]	-	device-key encrypt	●	●	●	●	○	●	-
NoCrack [20]	-	encode-then-encrypt	●	●	●	○	○	●	$\mathcal{F}_{rM3PM-IA}$
Amnesia [57]	-	generative	●	●	●	●	●	●	-
Kamouflage [14]	-	encode-then-encrypt	●	●	●	○	○	●	$\mathcal{F}_{rM3PM-IA}$
CSF-BPM [62]	hashed <i>mpw</i>	device-key encrypt	●	○	●	○	○	●	$\mathcal{F}_{rM3PM-EA}$
Tapas [38]	-	device-key encrypt	●	●	●	●	●	●	-

[†] ● : The PM fulfills this security property; ○ : The PM does not fulfill this security property. The ✗ in a column of provable security means this protocol has a security flaw. 1Password and Multipass achieve $\mathcal{F}_{adM3PM-EA}$. The “-” symbol in columns of protocol syntax means the PM does not use any cryptographic method, and the “-” symbol in a column of provable security means this protocol cannot be proven in the current model. It is worth noting that Enpass, Sticky PM, Password Boss, SafeInCloud, iCloud Keychain, and TeamPass do not have enough information details for determining the correctness of the documentation, so we analyze their M3PM protocols from documentation descriptions. We do not have enough information about Norton PM, Chrome PM, and Passworden for protocol analysis. BluePass [32], Amnesia [57], and Tapas [38] including additional devices are beyond our current model, and we leave these for future work.

* We divide offline guessing attacks into four types: Type-I refers to the adversary attacking the protocol without additional leaked information; Type-II refers to the adversary attacking the protocol with a corrupted server; Type-III refers to the adversary knowing partial leaked passwords (leaked by other websites) in the password vault; Type-IV refers to the adversary both corrupts the server and knows partial leaked passwords (leaked by other websites).

[‡] SRP+two-key means the user uses two secrets: a master password and a random string to authenticate and retrieve password vaults.

$H(mpw, salt)$. The ID_S uses the key mk' to decrypt the encrypted vault and verify the output. If the encryption algorithm is an authentication encryption, such as AES-GCM, then the wrong key leads to a failed decryption which eliminates a wrong guess. If the wrong key leads to a wrong plaintext instead of a failed decryption, the adversary can use the *GuessVerify* algorithm to determine the correctness of the plaintext. For example, if the decrypted result gets jumbled (e.g., containing non-ASCII characters), it is a wrong decryption with high probability. The overall offline guessing attack is formally described in Algo. 1.

Therefore, the *mpw encrypt* and *device-key encrypt* retrieval schemes cannot resist an offline guessing attack. What’s worse, the *encode-then-encrypt* mechanism which is designed to resist offline guessing attacks [14], [20] cannot

restrict the attack into negligible probability. The *encode-then-encrypt* algorithm relies on a distribution X to the decryption of a wrong key following the distribution X . If the adversary inputs the same distribution X into the *GuessVerify* algorithm, a wrong decrypted password will have a high probability of passing the *GuessVerify* algorithm. Therefore, the adversary needs to perform *OnlineTest*, then the offline guessing attack is transformed into a less threatening online guessing attack.

However, the distribution in the encoding algorithm cannot perfectly deny the *GuessVerify* algorithm, for the adversary can adjust its verification distribution adaptively. For example, if one credential decrypts to abc12345, which has a high probability in Zipf-distribution, but the corresponding website requires at least one special character

in passwords, then abc12345 has zero probability of being correct. This defect seemingly can be mitigated by adopting a more accurate distribution for each credential. However, the mitigation is endless, for the adversary may know one correct password in the vault and then the adversary could verify the guess with a high probability.

Generally, the generative M3PM can resist offline guessing, for the adversary cannot verify the guess offline without any value for verification and has to resort to online verification. Because all passwords generated by generative M3PM follow uniform distribution, the adversary cannot gain an advantage by gaining knowledge on distribution. However, if one generated password is leaked, the adversary gains the advantage of offline guessing and verifying the guess.

Algorithm 1: Offline guessing on password vaults.

Input: The encrypted password vault $eVault$, the expected random variable for master password X_{mpw} , the expected random variable for password X_{pwd} , and the guess threshold σ .
Output: The passwords vault $Vault$.

```

1  $Guess\_mpw\_set \leftarrow X_{mpw}.sortPr(order = descend);$ 
2 //Use  $X_{mpw}$  to generate a guess dictionary.
3  $K \leftarrow \epsilon;$  //Initialize a variable.
4  $Vault \leftarrow \phi;$ 
5 while ! $Guess\_mpw\_set.empty()$  do
6    $Guess\_mpw \leftarrow Guess\_mpw\_set.pop();$ 
7    $Guess\_mk \leftarrow KDF(Guess\_mpw);$  //Derive the key by
   following the specific protocol.
8    $Guess\_vault \leftarrow Dec(Guess\_mk, eVault);$ 
9    $\delta \leftarrow GuessVerify(Guess\_vault, X_{pwd}, \sigma)$ 
10  if  $\delta = 1$  then
11     $pwd' \leftarrow Guess\_vault[login];$  //Select one credential.
12    if  $OnlineTest(login, pwd')$  then
13       $K \leftarrow Guess\_mpw;$  //If the credential logs in
      successfully, the vault is decrypted.
14       $Vault \leftarrow Guess\_vault;$ 
15      return  $Vault;$ 
16 return  $Vault$ 

```

In Table 1, we summarize the results of our security analysis on offline guessing. We categorize offline guessing into four types depending on the ability of the adversary. Type-I adversaries can interact with the protocol but have no additional information about users. All PMs can resist this type of adversary because the adversary has no way to verify their guesses. Type-II adversary can corrupt the server and learn the internal state of the corrupted server. 1Password and Multipassword adopt two secrets, i.e., master password and random secret key, to resist a guessing attack. Lesspass, Gokey, SPHINX [48], and Amnesia [57] do not store any value about password vaults. NoCrack [20] and Kamouflage [14] use the *encode-then-encrypt* method to resist offline guessing upon leakage. BluePass [32] and Tapas [38] use another device, and only a corrupted server cannot give the adversary the ability to guess the password.

In Type-III, the adversary learns partially leaked passwords in password vaults, and LessPass and Gokey suffer from this kind of adversary. The adversary can guess the master password and generate the password for verification. If the adversary can learn partially leaked passwords and password vaults, i.e., a Type-IV adversary, only another secret (1Password and Multipassword) or additional device (BluePass and Tapas) can resist this offline guessing attack.

Insights. Our results show that all PMs capable of resisting offline guessing attacks introduce another high-entropy secret. For instance, 1Password and Multipassword require users to keep an additional secret string, while BluePass [32], Amnesia [57], and Tapas [38] introduce another device to store secret keys. This approach makes sense because if a password vault is protected by a single master password, the master password alone cannot provide sufficient entropy for protection, and some verifier, such as password vaults or generated passwords, can be obtained by the adversary. Therefore, without an additional high-entropy secret, M3PM protocols heavily rely on technologies such as slow hash or memory-hard functions.

5.2. Metadata Privacy

In the works of Gasti et al. [24] and Oesch and Ruoti [41], the assessment of metadata privacy focuses on the encryption of metadata and the encryption algorithms used. Overall, 30 of the 43 PMs have unencrypted metadata (see Table 1), which violates the principles of indistinguishability and non-malleability of metadata (see details in Appendix E of the full paper at <https://bit.ly/3zBPVUO>).

The malleability of metadata has more severe negative effects than that of secret passwords. While password malleability primarily results in usability issues, such as users being unable to log in with tampered passwords, an adversary could change a website's URL to a phishing URL. If users unknowingly access phishing websites, PMs might autofill passwords into these sites. Additionally, PMs often provide click-to-jump features for domains, allowing users to navigate to the domain with a click. This feature makes phishing attacks even more dangerous.

5.3. Case Study 1: 1Password

We show the details of the M3PM protocol of 1Password in Fig. 5. 1Password adopts SRP [59] for authentication, and *device-key encrypt* for password-processing. According to the above analysis, SRP and *device-key encrypt* cannot resist offline guessing attacks under corruption. However, 1Password adopts a two-key mechanism in that users remember a master password mpw and save a 44-character random string called secret key $SecretK$. Both authenticator $x = H_1(mpw, ID_C) \oplus H_1(SecretK, ID_C)$ and encryption key $mk = H_2(mpw, ID_C) \oplus H_2(SecretK, ID_C)$ are derived by the combination of the master password and secret key so that the adversary cannot conduct offline guessing attack without the knowledge of the secret key.

Before our proof, we establish some necessary assumptions. First, we assume that the hash functions in the protocol are modeled as random oracles, the public encryption algorithm satisfies IND-CPA, and symmetric encryption uses an AEAD algorithm. Additionally, while the SRP protocol lacks a formal security proof, no security flaws have been identified to date [60]. Our work does not aim to address the challenge of proving SRP's security. Therefore, we use a functionality $\mathcal{F}_{pwAuth}$ based on the work of Canetti et al. [19] and Groce and Katz [25] to replace the SRP protocol. Details of preliminary assumptions are in Appendix A.

Protocol OP

Input: In the initialization phase, the client inputs a master password $mpw \in \text{Zipf}(n)$, a secret key $\text{Secret}K \in \{0, 1\}^n$, login metadata login , and password pwd . In the retrieval phase, the client inputs a master password mpw' , a secret key $\text{Secret}K'$, and login metadata login . The server inputs nothing; the common input is the security parameter n .

Output: The client outputs a password pwd in the initialization and retrieval phases. The server outputs a success/failure.

Protocol:

I Initialization Phase.

- (a) [Registration] The client ID_C utilizes master password mpw and secret key $\text{Secret}K$ to generate a master authenticator $x = H_1(mpw, ID_C) \oplus H_1(\text{Secret}K, ID_C)$ and a master key $mk = H_2(mpw, ID_C) \oplus H_2(\text{Secret}K, ID_C)$. Subsequently, the client generates a vault key vk and private key sk , and initializes an empty vault $Vault$. The client employs the mk to encrypt the private key $esk = \text{AE.Enc}_{mk}(sk)$, utilizes the private key sk to encrypt the vault key $evk = \text{PKE.Enc}_{sk}(vk)$, and finally, uses the vault key vk to encrypt the vault $eVault = \text{AE.Enc}_{vk}(Vault)$. If there is no record $(\text{FILE}, ID_C, ID_S, x)$ in ID_S , ID_C queries $(\text{STOREPWDFILE}, sid, ID_C, x)$ to the functionality $\mathcal{F}_{pwAuth}$, and transmits $(evk, esk, eVault)$ to the server.
- (b) [Authentication] The client sends $(\text{NEWSESSION}, sid, ID_C, ID_S, x, \text{Client})$ to $\mathcal{F}_{pwAuth}$, and the server sends $(\text{NEWSESSION}, sid, ID_S, ID_C, \text{Server})$ to $\mathcal{F}_{pwAuth}$.
- (c) [Vault initialization] Upon receiving SUCCESS from $\mathcal{F}_{pwAuth}$, the server sends $(\text{INIT}, evk, esk, eVault)$ to the client. The client uses the mk to decrypt the key and vault. Subsequently, the client initializes the password pwd with the login metadata login . Finally, the client outputs the pwd , re-encrypts the vault, vault key and secret key $eVault, evk, esk$, and sends $(eVault, evk, esk)$ to the server.

II Retrieval Phase.

- (a) [Authentication] The client and server use the same authentication method in the initialization phase.
- (b) [Vault retrieval] Upon receiving SUCCESS from $\mathcal{F}_{pwAuth}$, the server sends $(\text{RETR}, evk, esk, eVault)$ to the client. Then, the client uses the master key mk to decrypt the key and vault. Finally, the client uses login metadata login to retrieve the password pwd and outputs it.

Figure 5: The complete M3PM protocol of 1Password. The client and server execute the SRP protocol with master authenticator $x = H_1(mpw, ID_C) \oplus H_1(\text{Secret}K, ID_C)$. We use functionality $\mathcal{F}_{pwAuth}$ in this protocol.

Theorem 1. *Let the client and server use an authentication sub-protocol UC-realizing $\mathcal{F}_{pwAuth}$. Let H_1, H_2 be random oracles, $\text{PKE} = (\text{PKE.KGen}, \text{PKE.Enc}, \text{PKE.Dec})$ be an IND-CPA-secure public key encryption scheme, and $\text{AE} = (\text{AE.KGen}, \text{AE.Enc}, \text{AE.Dec})$ be an authenticated encryption scheme. Let OP be the M3PM protocol of 1Password in Fig. 5. Then the OP protocol UC-realizes the $\mathcal{F}_{adM3PM-EA}$ of Fig. 4 in $\mathcal{F}_{pwAuth}$ -hybrid model.*

PROOF SKETCH. The proof can be found in Appendix C. Here we provide a brief sketch of the main idea in the proof.

We define a series of games [49] from the real-world execution to ideal-world execution and prove that the environment $\mathcal{Z}$ can distinguish each two games in a negligible probability, which means the environment cannot distinguish the real world and the ideal world. The simulator for the M3PM protocol of 1Password is depicted in Fig. 8.

The challenge of constructing a simulator is that, without knowledge of the correct master password, the simulator cannot determine whether the master password sent by the adversary is correct. To address this, the simulator records every initialization query sent by the adversary and rejects any retrieval query from the adversary that lacks a corresponding initialization. This solution is effective because the probability of the adversary correctly guessing the authenticator is negligible.

The simulator records every initialization by an honest

client and stores a “garbage” vault and a “garbage” key, which are random values. This storage maintains consistency, ensuring that the adversary can verify the consistency of an honest client’s storage without gaining any knowledge or tampering with it. For every encrypted vault and key sent by the adversary, the simulator checks whether the vault has been previously stored (leveraging the INT-CTXT security reduction for authenticated encryption). If a stored vault exists, the simulator encrypts a “garbage” vault and sends it to the adversary (leveraging the IND-CPA security reduction for authenticated encryption).

Remark 3. We briefly mention the proof for other schemes. Without an additional high-entropy secret key, the adversary has a non-negligible chance of correctly guessing the master password. Therefore, the simulator needs to program the random oracle and record the queries for any master passwords. Then, the simulator uses the TESTMPW query to test the correctness of the master password. By doing so, the simulator can respond correctly to the adversary.

5.4. Case Study 2: Passbolt

The details of Passbolt’s M3PM protocol are depicted in Fig. 6. Passbolt uses *pk challenge* for authentication, and *device-key encrypt* for retrieval. Specifically, Passbolt employs the OpenPGP standard [29] for encryption, introducing a public key pair. The client stores an encrypted secret key esk , and both the client and server have the

Protocol PB

Input: In the initialization phase, the client inputs a master password $mpw \in \text{Zipf}(n)$, login metadata $login$, and password pwd . In the retrieval phase, the client inputs a master password mpw' , login metadata $login$.

Output: The client outputs a password pwd in the initialization and retrieval phases. The server outputs a success/failure.

Protocol:

I Initialization Phase.

- (a) [Registration-Auth] The client ID_C generates a public key pair and sends the public key pk_C to the server. Subsequently, the client utilizes the master password to encrypt the private key $esk = AE.Enc_{H(mpw)}(sk_C)$.
- (b) [Registration-Vault] The client ID_C generates a vault key $vk \leftarrow \{0, 1\}^n$ and encrypts it with private key $evk = PKE.Enc_{pk_C}(vk)$. Finally, the client uses the vault key to encrypt an empty vault $eVault = AE.Enc_{vk}(Vault)$, sending $evk, eVault$ to the server.
- (c) [Authentication] The client sends $kf = H(pk_C)$ to start the session, and the server checks $kf \stackrel{?}{=} H(pk_C)$. Then the server generates a random value $b \leftarrow \{0, 1\}^n$ and utilizes the public key to challenge the client $cha = PKE.Enc_{pk_C}(b)$. If the client can decrypt and answer the b correctly, then authentication is successful.
- (d) [Vault initialization] If the server authenticates the client, then the server sends $(INIT, eVault, evk)$ to the client. Then, the client uses the master key $mk = H(mpw)$ to decrypt the key and vault. Subsequently, the client can initialize the password pwd with the login metadata $login$. Finally, the client outputs the password pwd , re-encrypts the vault $eVault'$ with new key vk' , and sends $eVault', evk'$ to the server.

II Retrieval Phase.

- (a) [Authentication] The client and server use the same authentication method in the initialization phase.
- (b) [Vault retrieval] If the server authenticates the client, then the server sends $(RETR, eVault, evk)$ to the client. Then the client uses the master key $mk = H(mpw)$ to decrypt the key and vault. Finally, the client uses login metadata $login$ to retrieve the password pwd , and outputs it.

Figure 6: The complete M3PM protocol of Passbolt. The compromised server has the capability to transmit the encrypted vault key evk as $cha = PKE.Enc_{pk_C}(vk)$ instead of sampling a new challenge to obtain the vault key $b' = vk$.

public key pk_C of the client ID_C . The server uses the public key pk_C and samples a random nonce $b \leftarrow \{0, 1\}^n$ to challenge the client for authentication $cha = PKE.Enc_{pk_C}(b)$. Then, the client uses the hash of master password $H(mpw)$ to decrypt the encrypted secret key $sk_C = AE.Dec_{H(mpw)}(esk)$ and uses the secret key to respond to the challenge. The secret key sk is also used to decrypt the vault key $vk = PKE.Dec_{sk_C}(evk)$ and the vault key is sampled freshly in each encryption.

The protocol of Passbolt has an oracle attack in that the corrupted server ID_S can learn the vault key vk of the client ID_C . The security flaw arises because the secret key sk_C is also used to decrypt both the challenge and the encrypted vault key. The corrupted server sends the encrypted vault key evk as challenge $cha' = evk$ instead of sampling a new random nonce. Then the honest client will decrypt the challenge and respond with the vault key to the server $evk' = PKE.Dec_{sk_C}(cha)$. The client uses sk_C to answer the challenge, which gives the adversary an oracle to use sk_C for decryption. Despite the client updating the vault key in each initialization, the server can gain the vault key in each authentication of the protocol.

During our disclosure, the security team at Passbolt explained that in practice, the mechanism is secure because the nonce used is concatenated with padding, and the client side performs format checking. However, we pointed out to them that their nonce format checking could still be vulnerable to padding oracle attacks [13], as the public key

encryption used by OpenPGP follows the PKCS#1 v1.5 standard, which is not CCA-secure. The core problem with this protocol is that the same secret key is used both to respond to the challenge and to decrypt the vault key. Given that Passbolt introduces a public key pair, we recommend implementing an authenticated key exchange protocol, such as HMQV [30], to resist the oracle attack.

6. Discussion

6.1. Mitigation on Offline Guessing Attacks

We now discuss three promising methods to mitigate offline guessing attacks: slow hash/memory-hard function, password policy, and honey encryption.

Slow hash and memory-hard function. One practical approach to weakening the effectiveness of offline guessing attacks is to slow down the speed of each guess, thereby limiting the number of guesses an adversary can perform within a given period. This can be achieved through the use of slow hash functions and memory-hard functions as key derivation functions (KDFs). Slow hash functions deliberately increase the computational time required for each guess, making it more difficult for adversaries to perform a high number of iterations quickly. Memory-hard functions, on the other hand, require substantial memory resources in addition to computational time, further hindering the adversary's ability to efficiently carry out offline guessing attacks. Both of these

methods are employed to effectively slow down step 6 in Algo. 1, thereby enhancing the security of the password managers against offline guessing attempts.

Blocki et al. [12] analyzed the strategies and capabilities of offline guessing attackers, finding that slow hash functions are ineffective against offline guessing attacks, whereas memory-hard functions are more effective. In our research, we consider these “slowing” strategies as mechanisms that effectively expand the password guess space. This is because the time required for one guess with these strategies might be equivalent to 2^{15} guesses without them. However, it is crucial to note that these strategies do not affect the password distribution. In simpler terms, if users choose a weak master password like 123456, the effectiveness of slow hash and memory-hard functions is limited.

Password creation policy. Another way to weaken the ability of an offline guessing attack is to reduce the probability of guessing the correct master password, thereby forcing the adversary to conduct more guesses. PMs adopt different password creation policies for master passwords. For example, LastPass requires master passwords to be longer than 12 characters and have at least one lowercase letter, one uppercase letter, and one digit, while Dashlane does not set explicit requirements. Though there is no research showing which password creation policy benefits most, we believe that adopting a password creation policy and blocking simple passwords, such as 123456 and password, can mitigate offline guessing attacks.

Honey encryption. We now discuss honey encryption [20], a specific encryption algorithm used in the *encode-then-encrypt* password-processing method. The encoding algorithm is designed to render the *GuessVerify* algorithm ineffective, forcing the adversary to rely on online verification, which is less efficient and more likely to be detected by websites. As examined in Sec. 5.1, while the encoding process cannot reduce the probability of a correct guess by *GuessVerify* to a negligible level, it increases the adversary’s knowledge requirements about users’ passwords.

However, honey encryption presents other challenges. Honey encryption struggles to achieve indistinguishability and non-malleability of metadata. Indeed, if the client employs a key derived from the master password to encrypt the metadata, honey encryption would also need to operate on the metadata. The distribution of the metadata needs to be encoded, which introduces an additional layer of complexity and considerations for honey encryption in scenarios. Moreover, neither metadata nor secret passwords can achieve non-malleability because the honey encryption mechanism never indicates a failure, even with incorrect decryption.

Another challenge lies in integrating honey encryption with explicit authentication. If the server stores an authenticator for the master password, it becomes vulnerable to offline guessing attacks in the event of a server compromise. Therefore, an authenticator for explicit authentication undermines the effectiveness of honey encryption.

6.2. Recommendations

According to our analysis in Sec. 5.1, all the explicit authentication schemes in Fig. 2 cannot resist offline guessing attacks upon server compromise. In essence, for authentication with only one master password, the server has to store such an authenticator because this is the only secret between the user and the server. However, the low entropy of the master password makes the authenticator vulnerable to guessing attacks. Therefore, we recommend PMs adopt two-factor authentication. We highlight these two factors must be incorporated into authentication and encryption.

A direct and low-cost method to address the problem of guessing attacks is to introduce another high-entropy secret into the protocols, similar to 1Password and Multipassword. This additional secret may initially seem like a management burden for users, but proper policies can mitigate this. For example, 1Password advises users to store the additional secret key on a piece of paper and keep it safe. Typically, users do not need to use the secret key frequently, as it is stored on their primary device. Only when using PMs on a new device do users need to enter this secret key.

Given its significant security benefits, the additional secret does not impose a substantial usability burden. Furthermore, this method allows users to maintain simple, easy-to-remember passwords without compromising security. The high-entropy secret significantly enhances security against offline guessing attacks, as it adds a layer of complexity that adversaries must overcome. Consequently, the approach reduces the reliance on computationally expensive techniques like slow hashing or memory-hard functions. Overall, the introduction of a high-entropy secret strikes a balance between security and usability, providing robust protection against various attack vectors while maintaining convenience.

6.3. Future Works

In addition to M3PM protocols, PMs incorporate various other sub-protocols, such as password reset, password sharing, two-factor authentication protocols, and more. Analyzing these sub-protocols involves gathering sufficient information to describe each protocol accurately and designing new functionalities to formalize them, which may introduce new entities, such as hardware/media for storing. We leave the detailed analysis of other sub-protocols as future work.

7. Conclusion

In this paper, we present a systematic security analysis of master-password-protected password management (M3PM) protocols in password managers. For the first time, we identify M3PM protocols from 43 password managers from both academia and industry, define their functionalities in the UC framework, and conduct a security analysis. Our analysis reveals several security issues in current M3PM protocols, and we provide recommendations to address these issues. Given the crucial role of password managers, we believe our work significantly advances research on their protocols.

Acknowledgments

The authors are grateful to the shepherd and anonymous reviewers for their invaluable comments. Ding Wang is the corresponding author. This research was in part supported by the National Natural Science Foundation of China under Grants Nos. 62222208 and 62172240, and by the Fundamental Research Funds for the Central Universities, Nankai University (Grant No. 63243154). See the full version of this paper at <https://bit.ly/3zBPVUO>.

References

- [1] *Password managers: using browsers and apps to safely store your passwords*, Dec. 2021, <https://www.ncsc.gov.uk/collection/top-tips-for-staying-secure-online/password-managers>.
- [2] *Why the password isn't dead quite yet*, Jul. 2021, <https://arstechnica.com/information-technology/2021/07/why-the-password-isnt-dead-quite-yet/>.
- [3] *NIST SP 800-63: Digital Identity Guidelines Frequently Asked Questions*, Mar. 2022, <https://pages.nist.gov/800-63-FAQ/>.
- [4] *The Password Isn't Dead Yet. You Need a Hardware Key*, 2022, <https://www.wired.com/story/hardware-security-key-passwords-passkeys/>.
- [5] *Have I been pwned?*, 2023, <https://haveibeenpwned.com/>.
- [6] *LastPass Technical Whitepaper*, Jun. 2024, <https://support.lastpass.com/download/lastpass-technical-whitepaper>.
- [7] M. Abdalla, M. Barbosa, T. Bradley, S. Jarecki, J. Katz, and J. Xu, "Universally composable relaxed password authenticated key exchange," in *Proc. CRYPTO 2020*, vol. 12170, pp. 278–307.
- [8] L. Abrams, *LastPass breach linked to theft of \$4.4 million in crypto*, 2023, <https://www.bleepingcomputer.com/news/security/lastpass-breach-linked-to-theft-of-44-million-in-crypto/>.
- [9] N. Alkaldi and K. Renaud, "Why do people adopt, or reject, smartphone password managers?" in *Proc. EuroUSEC 2016*.
- [10] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE EuroS&P 2016*, pp. 292–302.
- [11] Bitwarden, *Bitwarden Security Whitepaper - Vault Data*, 2023, <https://bitwarden.com/help/vault-data/>.
- [12] J. Blocki, B. Harsha, and S. Zhou, "On the economics of offline password cracking," in *Proc. IEEE S&P 2018*, pp. 35–53.
- [13] H. Böck, J. Somorovsky, and C. Young, "Return of bleichenbacher's oracle threat (ROBOT)," in *Proc. USENIX SEC 2018*, pp. 817–849.
- [14] H. Bojinov, E. Bursztein, X. Boyen, and D. Boneh, "Kamouflage: Loss-resistant password management," in *Proc. ESORICS 2010*.
- [15] J. Bonneau, "The science of guessing: Analyzing an anonymized corpus of 70 million passwords," in *Proc. IEEE S&P 2012*.
- [16] J. Bonneau, C. Herley, P. van Oorschot, and F. Stajano, "Passwords and the evolution of imperfect authentication," *Commun. ACM*, vol. 58, no. 7, pp. 78–87, 2015.
- [17] J. Bonneau, C. Herley, P. C. Van Oorschot, and F. Stajano, "The quest to replace passwords: A framework for comparative evaluation of web authentication schemes," in *Proc. IEEE S&P 2012*, pp. 553–567.
- [18] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. FOCS 2001*, pp. 136–145.
- [19] R. Canetti, S. Halevi, J. Katz, Y. Lindell, and P. D. MacKenzie, "Universally composable password-based key exchange," in *Proc. EUROCRYPT 2005*, vol. 3494, pp. 404–421.
- [20] R. Chatterjee, J. Bonneau, A. Juels, and T. Ristenpart, "Cracking-resistant password vaults using natural language encoders," in *Proc. IEEE S&P 2015*, pp. 481–498.
- [21] Y. Choong, "A cognitive-behavioral framework of user password management lifecycle," in *Proc. HAS 2014*, pp. 127–137.
- [22] G. T. Davies, S. H. Faller, K. Gellert, T. Handirk, J. Hesse, M. Horváth, and T. Jager, "Security analysis of the whatsapp end-to-end encrypted backup protocol," in *Proc. CRYPTO*, 2023.
- [23] A. Gangwal, S. Singh, and A. Srivastava, "Autospill: Credential leakage from mobile password managers," in *Proc. CODASPY 2023*.
- [24] P. Gasti and K. B. Rasmussen, "On the security of password manager database formats," in *Proc. ESORICS 2012*, pp. 770–787.
- [25] A. Groce and J. Katz, "A new framework for efficient password-based authenticated key exchange," in *Proc. ACM CCS 2010*, pp. 516–525.
- [26] A. Hanamsagar, S. S. Woo, C. Kanich, and J. Mirkovic, "Leveraging semantic transformation to investigate password habits and their causes," in *Proc. ACM CHI 2018*, pp. 1–10.
- [27] I. Ion, R. Reeder, and S. Consolvo, "'...No one can hack my mind': Comparing expert and non-expert security practices," in *Proc. SOUPS 2015*, pp. 327–346.
- [28] S. Jarecki, H. Krawczyk, and J. Xu, "OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks," in *Proc. EUROCRYPT 2018*, vol. 10822, pp. 456–486.
- [29] C. Jon, D. Lutz, F. Hal, S. Daphne, and T. Rodney, "Openpgp message format," Nov. 2007, <https://data.ietf.org/doc/html/rfc4880>.
- [30] H. Krawczyk, "HMQV: A high-performance secure diffie-hellman protocol," in *Proc. CRYPTO 2005*, vol. 3621, pp. 546–566.
- [31] LastPass, *An Encryption Model that Prioritizes your Security*, 2023, <https://www.lastpass.com/security/zero-knowledge-security>.
- [32] Y. Li, H. Wang, and K. Sun, "Bluepass: A secure hand-free password manager," in *Proc. SecureComm 2017*, pp. 185–205.
- [33] Z. Li, W. He, D. Akhawe, and D. Song, "The emperor's new password manager: Security analysis of web-based password managers," in *Proc. USENIX SEC 2014*, pp. 465–479.
- [34] J. Ma, W. Yang, M. Luo, and N. Li, "A study of probabilistic password models," in *Proc. IEEE S&P 2014*, pp. 689–704.
- [35] P. Mayer, C. W. Munyendo, M. L. Mazurek, and A. J. Aviv, "Why users (don't) use password managers at a large educational institution," in *Proc. USENIX SEC 2022*, pp. 1849–1866.
- [36] P. Mayer, Y. Zou, F. Schaub, and A. J. Aviv, "'Now i'm a bit angry': Individuals' awareness, perception, and responses to data breaches that affected them," in *Proc. USENIX SEC 2021*, pp. 393–410.
- [37] D. McCarney, "Password managers: Comparative evaluation, design, implementation and empirical analysis," Master's thesis, Carleton University, 2013.
- [38] D. McCarney, D. Barrera, J. Clark, S. Chiasson, and P. C. van Oorschot, "Tapas: Design, implementation, and usability evaluation of a password manager," in *Proc. ACM ACSAC 2012*, pp. 89–98.
- [39] C. W. Munyendo, P. Mayer, and A. J. Aviv, "'i just stopped using one and started using the other': Motivations, techniques, and challenges when switching password managers," in *Proc. ACM CCS 2023*.
- [40] NordPass, *People have around 170 passwords on average, study shows*, 2024, <https://www.globenewswire.com/en/news-release/2024/05/21/2885556/0/en/People-have-around-170-passwords-on-average-study-shows.html>.
- [41] S. Oesch and S. Ruoti, "That was then, this is now: A security evaluation of password generation, storage, and autofill in browser-based password managers," in *Proc. USENIX SEC 2020*.
- [42] B. Pal, T. Daniel, R. Chatterjee, and T. Ristenpart, "Beyond credential stuffing: Password similarity models using neural networks," in *Proc. IEEE S&P 2019*, pp. 417–434.
- [43] S. Pearman, J. Thomas, P. E. Naeini, H. Habib, L. Bauer, N. Christin, L. F. Cranor, S. Egelman, and A. Forget, "Let's go in for a closer look: Observing passwords in their natural habitat," in *Proc. ACM CCS 2017*, pp. 295–310.

- [44] S. Pearman, S. Zhang, L. Bauer, and N. Christin, “Why people (don’t) use password managers effectively,” in *Proc. SOUPS 2019*.
- [45] H. Ray, F. Wolf, R. Kuber, and A. J. Aviv, “Why older adults (don’t) use password managers,” in *Proc. USENIX SEC 2021*, pp. 73–90.
- [46] B. Reviews, *Which Password Managers Have Been Hacked?*, 2023, <https://password-managers.bestreviews.net/faq/which-password-managers-have-been-hacked/>.
- [47] B. Ross, C. Jackson, N. Miyake, D. Boneh, and J. C. Mitchell, “Stronger password authentication using browser extensions,” in *Proc. USENIX SEC 2005*, pp. 17–31.
- [48] M. Shirvanian, N. Saxena, S. Jarecki *et al.*, “Building and studying a password store that perfectly hides passwords from itself,” *IEEE Trans. Dependable Secur. Comput.*, vol. 16, no. 5, pp. 770–782, 2019.
- [49] V. Shoup, “Sequences of games: a tool for taming complexity in security proofs,” *IACR Cryptol. ePrint Arch.*, 2004.
- [50] D. Silver, S. Jana, D. Boneh, E. Y. Chen, and C. Jackson, “Password managers: Attacks and defenses,” in *Proc. USENIX SEC 2014*.
- [51] K. Toubba, *Notice of Recent Security Incident: The LastPass Blog*, 2022, <https://blog.lastpass.com/2022/12/notice-of-recent-security-incident/>.
- [52] J. Wallen, *Six of the most popular Android password managers are leaking data*, 2023, <https://www.zdnet.com/article/six-of-the-most-popular-android-password-managers-are-leaking-data/>.
- [53] D. Wang, H. Cheng, P. Wang, X. Huang, and G. Jian, “Zipf’s law in passwords,” *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 11, pp. 2776–2791, 2017.
- [54] D. Wang, P. Wang, D. He, and Y. Tian, “Birthday, name and bifacial-security: Understanding passwords of Chinese web users,” in *Proc. USENIX SEC 2019*, pp. 1537–1555.
- [55] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang, “Targeted online password guessing: An underestimated threat,” in *Proc. ACM CCS 2016*, pp. 1242–1254.
- [56] D. Wang, Y. Zou, Y.-A. Xiao, S. Ma, and X. Chen, “PASS2EDIT: A multi-step generative model for guessing edited passwords,” in *Proc. USENIX SEC 2023*, pp. 983–1000.
- [57] L. Wang, Y. Li, and K. Sun, “Amnesia: A bilateral generative password manager,” in *Proc. IEEE ICDCS 2016*, pp. 313–322.
- [58] D. Wheeler, “zxcvbn: Low-budget password strength estimation,” in *Proc. USENIX SEC 2016*, pp. 157–173.
- [59] T. Wu, “A real-world analysis of kerberos password security,” in *Proc. NDSS 1999*, pp. 13–22.
- [60] T. Wu, “Srp-6: Improvements and refinements to the secure remote password protocol,” <http://srp.stanford.edu/srp6.ps>, 2002.
- [61] A. Zaharia, *Have I Been Hacked? How To Recognize & Recover From a Hack*, 2023, <https://www.aura.com/learn/have-i-been-hacked>.
- [62] R. Zhao and C. Yue, “All your browser-saved passwords could belong to us: A security analysis and a cloud-based new design,” in *Proc. CODASPY 2013*, pp. 333–340.
- [63] R. Zhao, C. Yue, and K. Sun, “Vulnerability and risk analysis of two commercial browser and cloud based password managers,” *ASE Science Journal*, vol. 1, no. 4, pp. 1–15, 2013.

Appendix A.

Basic Cryptographic Security Assumptions

IND-CPA security for public key encryption. For public key encryption $PKE = (\text{KGen}, \text{Enc}, \text{Dec})$, the indistinguishability of ciphertext under chosen plaintext attacks (IND-CPA) security refers to that adversary can choose plaintext for encryption but still cannot distinguish ciphertext. Formally, the adversary $\mathcal{A}$ can use the public

key pk generated by KGen for encryption, and give the challenger two messages m_0, m_1 . The challenger selects a bit $b \leftarrow \{0, 1\}$ randomly and encrypts one of the messages $c = \text{Enc}_{pk}(m_b)$. Upon receiving the ciphertext c , the adversary $\mathcal{A}$ tries to distinguish which message is encrypted. The advantage of the adversary $\mathcal{A}$ for a correct distinguish b' is defined as $\text{Adv}_{\mathcal{A}, PKE}^{\text{ind-cpa}}(n) = \Pr[b' = b] - \frac{1}{2}$.

IND-CPA security for authenticated encryption. For an authenticated encryption $AE = (\text{KGen}, \text{Enc}, \text{Dec})$, the indistinguishability of ciphertext under chosen plaintext attacks (IND-CPA) security refers to that adversary can choose plaintext for encryption but still cannot distinguish ciphertext. Formally, the adversary $\mathcal{A}$ is given an encryption oracle $\text{Enc}_K()$ where the key K is generated by KGen , and sends challenger two message m_0, m_1 . The challenger selects a bit $b \leftarrow \{0, 1\}$ randomly and encrypts one of the messages $c = \text{Enc}_K(m_b)$. Upon receiving the ciphertext c , the adversary $\mathcal{A}$ tries to distinguish which message is encrypted. The advantage of the adversary $\mathcal{A}$ for a correct distinguish b' is defined as $\text{Adv}_{\mathcal{A}, AE}^{\text{ind-cpa}}(n) = \Pr[b' = b] - \frac{1}{2}$.

INT-CTXT security for authenticated encryption. For authenticated encryption $AE = (\text{KGen}, \text{Enc}, \text{Dec})$, the integrity of ciphertext (INT-CTXT) security ensures that the adversary cannot create a new valid ciphertext. Formally, the adversary $\mathcal{A}$ is given an encryption oracle $\text{Enc}_K()$ where the key K is generated by KGen and gives the challenger a new ciphertext c which has not been generated by $\text{Enc}_K()$ oracle. The advantage of the adversary $\mathcal{A}$ for a correct forgery is defined as $\text{Adv}_{\mathcal{A}, AE}^{\text{int-ctxt}}(n) = \Pr[\text{Dec}_K(c) \neq \perp]$.

The functionality $\mathcal{F}_{pwAuth}$. We slightly change the functionality of Jarecki et al.’s [28] and Groce and Katz’s work [25], and give a functionality called $\mathcal{F}_{pwAuth}$. The details of $\mathcal{F}_{pwAuth}$ are depicted in Fig. 7.

Appendix B.

Security Functionality

Generally, $\mathcal{F}_{M3PM}$ maintains a lookup table for authentication and password retrieval. This lookup table, encoded in VAULT, contains the user’s identity ID_C , the server’s identity ID_S , the master password mpw , login metadata $login$, and the password pwd . The functionality $\mathcal{F}_{M3PM}$ includes both initialization and retrieval phases. Any queries for a non-existent record will be ignored by $\mathcal{F}_{M3PM}$.

Initialization phase. In initialization phase, a client ID_C and a server ID_S initialize a password pwd for one credential $login$. During initialization, the client sets a master password mpw for authentication in the retrieval phase.

A client ID_C starts a new initialization by calling INITC query with master password mpw , login metadata $login$, and password pwd . If the password pwd is ϵ , the $\mathcal{F}_{M3PM}$ will generate a random password $pwd \leftarrow \{0, 1\}^n$ for client, and mark the password GENERATED (IC.1). Then, $\mathcal{F}_{M3PM}$ records this initialization session and ensures only one record for each session (IC.2). Finally, the functionality sends a message to inform adversary $\mathcal{A}$ and server ID_S that the client has initialized a new password (IC.3).

Functionality $\mathcal{F}_{pwAuth}$

On input (STOREPWDFILE, sid, ID_C, pw) from ID_S (or $\mathcal{A}$ if ID_S is corrupted):

- If there is the first STOREPWDFILE for ID_C , record $\langle \text{FILE}, ID_C, ID_S, pw \rangle$ and mark it UNCOMPROMISED

On input (STEALPWDFILE, sid, ID_C) from $\mathcal{A}$:

- If there is no FILE record for ID_C , then return “no password file” to $\mathcal{A}$
- If there is FILE record marked UNCOMPROMISED, then mark it COMPROMISED
- If there is a record $\langle \text{OFFLINE}, pw \rangle$, then send pw to $\mathcal{A}$. Else, send “password file stolen” to $\mathcal{A}$

On input (OFFLINETESTPWD, sid, pw') from $\mathcal{A}$:

- If $\langle \text{FILE}, ID_C, ID_S, pw \rangle$ is marked COMPROMISED, then if $pw' = pw$, return CORRECT to $\mathcal{A}$; else WRONG
- Else record $\langle \text{OFFLINE}, pw \rangle$

On input (NEWSESSION, $sid, ID_C, ID_S, pw, \text{Client}$) from ID_C (or $\mathcal{A}$ if ID_C is corrupted):

- Store a record $(ID_C, ID_S, sid, pw, \text{Client})$ labeled FRESH
- Send (NEWSESSION, $sid, ID_C, ID_S, \text{Client}$) to the adversary $\mathcal{A}$

On input (NEWSESSION, $sid, ID_S, ID_C, \text{Server}$) from ID_S (or $\mathcal{A}$ if ID_S is corrupted):

- Retrieve record $\langle \text{FILE}, ID_C, ID_S, pw \rangle$
- If a $(ID_C, ID_S, sid, pw, \text{Client})$ exists, store $(ID_S, ID_C, sid, pw, \text{Server})$ labeled FRESH
- Send (NEWSESSION, $sid, ID_S, ID_C, \text{Server}$) to the adversary $\mathcal{A}$

On input (TESTPWD, sid, ID_C, pw') from the adversary $\mathcal{A}$:

- If there is no stored record $(ID_C, ID_S, sid, pw, \text{role})$ marked FRESH, then abort. Otherwise, retrieve it
- If $pw = pw'$, label the record COMPROMISED and reply $\mathcal{A}$ with CORRECT guess
- If $pw \neq pw'$, label the record INTERRUPTED and reply $\mathcal{A}$ with WRONG guess

On input (COMPLETEAUTH, sid, ID_S, ID_C) from adversary $\mathcal{A}$:

- Retrieve the record $(ID_S, ID_C, sid, pw, \text{Server})$, and if it is marked COMPLETED, then abort
- If the record is COMPROMISED, or either ID_C or ID_S are corrupted, send SUCCESS to ID_S and $\mathcal{A}$
- If the record is INTERRUPTED, send FAIL to ID_S and $\mathcal{A}$
- Mark the record $(ID_S, ID_C, sid, pw, \text{Server})$ as COMPLETED

Figure 7: Ideal functionality $\mathcal{F}_{pwAuth}$ for password-based authentication.

If the server ID_S agrees to incorporate into the session, the ID_S sends an INIT query to $\mathcal{F}_{M3PM}$. The functionality retrieves the record that the client has initialized, and gets the value of master password mpw , login metadata $login$, and password pwd (IS.1). If there exists such a record, then the $\mathcal{F}_{M3PM}$ records a tuple $\langle \text{VAULT}, ID_C, ID_S, mpw, login, pwd \rangle$ and makes sure that one client only has one password for credentials (IS.2). Finally, the $\mathcal{F}_{M3PM}$ informs adversary $\mathcal{A}$ that the server ID_S has incorporated into the session (IS.3).

We let adversary $\mathcal{A}$ determine when to complete the protocol for modeling the ability that adversary can interrupt the message [18], [22]. The adversary can send a COMPLETEINITC query to determine the finish of the client. The functionality $\mathcal{F}_{M3PM}$ searches for a record for client initialization. If $\mathcal{F}_{M3PM}$ finds this record, it means that the client is ready to finish the session, then the $\mathcal{F}_{M3PM}$ deletes the record for session completed (CIC.1). Then, the $\mathcal{F}_{M3PM-EA}$ functionality checks whether there are vault records for ID_C and ID_S . If there exists a record $\langle \text{VAULT}, ID_C, ID_S, mpw', *, * \rangle$, the functionality verifies the correctness of mpw (CIC.2). This step captures the practical scenario where, after registering a vault in password managers, the M3PM protocol with explicit authentication will authenticate the master password and

then initialize passwords.

Similarly, the adversary $\mathcal{A}$ can send a COMPLETEINIT query, and the functionality will store a record represented by the password vault (CIS.1), and the server ID_S checks whether the master password is correct (CIS.2).

Retrieval phase. In retrieval phase, the client ID_C retrieves the password pwd for credential $login$ with the cooperation of the server ID_S by inputting the correct master password mpw . If the wrong master password is input, the server ID_S will receive a FAIL message. The output of the functionality $\mathcal{F}_{M3PM}$ for a wrong password depends on whether the M3PM protocol has explicit or implicit authentication.

A client ID_C starts a new retrieval session by a RETRC query with master password mpw' and login metadata $login$. The functionality records this query and the master password mpw' and makes sure that only one session has started (RC.1). Then the functionality sends a message to inform adversary $\mathcal{A}$ and server ID_S that the client has started a new retrieval session (IC.2).

After the client ID_C finishes the RETRC query, the server ID_S can incorporate into the session sid by sending a RETRS query. The functionality $\mathcal{F}_{M3PM}$ first retrieves the record that the client has stored in RETRC query (RS.1). Then, the functionality checks for stored vault records, and

non-existence represents the failure of retrieval (RS.2). If there exists such a record, the functionality will append the correct master password mpw into the RETRC record for the further check (RS.3). The functionality sends the adversary $\mathcal{A}$ about whether the retrieval is a success (RS.4).

Same with the initialization, the adversary determines the completion of the retrieval session. The adversary sends a COMPLETETRC query to the functionality, and the functionality retrieves for the RETRC record. The functionality marks the RETRC record RETRIEVED to ensure this record be deleted after the client and server both retrieve it (CRC.1). After that, the functionality checks two master passwords mpw, mpw' . If they are equal, the functionality prepares to send the correct password. And the functionality $\mathcal{F}_{M3PM-EA}$ will alert the correct result to the $\mathcal{A}$ (CRC.2). Otherwise, the functionality responds with two options. For explicit authentication $\mathcal{F}_{M3PM-EA}$, the functionality returns a FAIL message (CRC.3(b)); for implicit authentication $\mathcal{F}_{M3PM-IA}$, the functionality returns a random password. If the password was generated by functionality, then return a uniformly distributed password, otherwise return a Zipf-distributed [53] password (CRC.3(a)). Finally, the functionality sends the result to the client or adversary (CRC.4).

The adversary can send a COMPLETETRRS query. First, the functionality retrieves the RETRC record and does the same process as COMPLETETRC query (CRS.1). The response of the functionality is simple: if the mpw is checked successfully, then return SUCC to the server ID_S (CRS.2), otherwise return FAIL to the server (CRS.3). We allow the adversary to have a TESTMPW query in $\mathcal{F}_{M3PM}$. If the guess mpw is correct, the functionality sends $\mathcal{A}$ the correct pwd , otherwise the functionality generates a wrong pwd' which is a random value or FAIL (TM.1).

Appendix C. Proof of 1Password

Proof. Let $Real_{\mathcal{Z}, \mathcal{A}, OP}$ be the probability that environment $\mathcal{Z}$ interacting with adversary $\mathcal{A}$ and real-world protocol OP thinks it interacts with the real-world protocol, i.e., output 1, and let $Ideal_{\mathcal{Z}, SIMOP, \mathcal{F}_{adM3PM-EA}}$ be the probability that environment $\mathcal{Z}$ interacting with simulator $SIMOP$ and ideal-world functionality $\mathcal{F}_{adM3PM}$ outputs 1.

Game G_0 : This game is in the real world and the adversary $\mathcal{A}$ interacts with the real players following the protocol OP .

$$Real_{\mathcal{Z}, \mathcal{A}, OP} = \Pr[G_0] \quad (1)$$

Game G_1 : (Simulate real world). In this game, we perform simulation on the real players to make the adversary $\mathcal{A}$ and environment $\mathcal{Z}$ interact with the simulator. The simulation in this game is exactly the same as the real game, except the simulator stores records to simulate random oracles and all transcripts between the client ID_C and the server ID_S . This change is just for turning the real game into a game in the UC framework with only syntactical changes.

$$\Pr[G_1] = \Pr[G_0] \quad (2)$$

Game G_2 : (Record initialization). In this game, the simulator acts like $\mathcal{F}_{pwAuth}$ to $\mathcal{A}$ to keep the consistency of the $\mathcal{A}$ acting like a normal user. And the simulator stores the record from $\mathcal{F}_{adM3PM-EA}$ for consistency. For COMPLETEAUTH query, the simulator check whether this query is made from $\mathcal{F}_{adM3PM-EA}$ or $\mathcal{A}$ by checking the record. All changes in this game are internal and unobservable to the environment, so it is the same with G_1 .

$$\Pr[G_2] = \Pr[G_1] \quad (3)$$

Game G_3 : (Random key for incorrect guess). In this game, the simulator records $\langle MFILE, ID_C, ID_S, x \rangle$ for the adversary. For any (OFFLINETESTPWD, sid, x) query, the simulator looks for the $\langle MFILE, ID_C, ID_S, x \rangle$. If such a record exists, it returns CORRECT to $\mathcal{A}$. Otherwise, it returns WRONG. For any corrupted password files on the server, the simulator generates a random value x for the adversaries. In this game, the probability of distinguishing between G_1 and G_2 equals that the adversary correctly guesses x .

In our work, we assume passwords follow a Zipf distribution [53], so the probability of correctly guessing passwords is bounded by $C' \cdot (Q_{init} + Q_{retr})^{s'}$, where C' and s' are parameters of the Zipf distribution. We also consider the $SecretK$, determined by the security parameter n , as a random n -length string. The probability that the environment $\mathcal{Z}$ can distinguish between G_2 and G_3 is determined by the likelihood of simultaneously guessing the master password mpw and $SecretK$, which is negligible.

$$|\Pr[G_3] - \Pr[G_2]| \leq C' \cdot (Q_{init} + Q_{retr})^{s'} \cdot \frac{1}{2^n} \quad (4)$$

Game G_4 : (Unless vault key suite). In this game, the simulator replaces the encrypted secret key esk and vault key evk with an encryption of 0. We reduce distinguishing G_4 and G_3 on the IND-CPA security of authentication encryption AE and PKE . In G_3 , no adversary can guess both mpw and $SecretK$ correctly. Therefore, the master key mk is random and uniform for the adversary.

If there exists an adversary $\mathcal{D}$ can break IND-CPA of AE with probability $\text{Adv}_{\mathcal{D}, AE}^{\text{ind-cpa}}(n)$, then the environment $\mathcal{Z}$ can use the adversary $\mathcal{D}$ to distinguish G_4 and G_3 within probability $\text{Adv}_{\mathcal{D}, AE}^{\text{ind-cpa}}(n)$. Similarly, if there exists an $\mathcal{D}'$ can break IND-CPA of PKE with probability $\text{Adv}_{\mathcal{D}', PKE}^{\text{ind-cpa}}(n)$, then the environment $\mathcal{Z}$ can use the adversary $\mathcal{D}$ to distinguish G_4 and G_3 within probability $\text{Adv}_{\mathcal{D}, PKE}^{\text{ind-cpa}}(n)$.

$$|\Pr[G_4] - \Pr[G_3]| \leq \text{Adv}_{\mathcal{D}, AE}^{\text{ind-cpa}}(n) + \text{Adv}_{\mathcal{D}', PKE}^{\text{ind-cpa}}(n) \quad (5)$$

Game G_5 : (Client encrypts “garbage”). In this game, the client encrypts 0 instead of password vaults. We reduce distinguishing G_5 and G_4 on the IND-CPA security of authentication encryption AE . If there exists an adversary $\mathcal{D}$ can break IND-CPA of AE with probability $\text{Adv}_{\mathcal{D}, AE}^{\text{ind-cpa}}(n)$, then the environment $\mathcal{Z}$ can use the adversary $\mathcal{D}$ to distinguish G_3 and G_2 within probability $Q_{init} \cdot \text{Adv}_{\mathcal{D}, AE}^{\text{ind-cpa}}(n)$ where Q_{init} is the times of initialization.

Simulator SIM_{OP}

On input $(INITC, sid, ID_C, ID_S)$ from $\mathcal{F}_{adM3PM-EA}$:

- If there is no record $\langle VAULT, ID_C, ID_S, *, *, * \rangle$, then generate random value $eVault, evk, esk$, store $\langle VAULT, ID_C, ID_S, eVault, evk, esk \rangle$, and send $(INIT, eVault, evk, esk)$ to $\mathcal{A}$
- Send $(NEWSESSION, sid, ID_C, ID_S, Client)$ to $\mathcal{A}$

On input $(INITS, sid, ID_C, ID_S)$ from $\mathcal{F}_{adM3PM-EA}$:

- If there is no record $\langle VAULT, ID_C, ID_S, *, *, * \rangle$, then generate random value $eVault, evk, esk$, and store $\langle VAULT, ID_C, ID_S, eVault, evk, esk \rangle$, and send $(NEWSESSION, sid, ID_S, ID_C, Server)$ to $\mathcal{A}$

On input $(RETRC/RETRS, sid, ID_C, ID_S)$ from $\mathcal{F}_{adM3PM-EA}$:

- For RETRC, store a record $(RETRC, sid, ID_C, ID_S)$, and send $(NEWSESSION, sid, ID_C, ID_S, Client)$ to $\mathcal{A}$
- For RETRS, store a record (sid, ID_C, ID_S, res) , and send $(NEWSESSION, sid, ID_S, ID_C, Server)$ to $\mathcal{A}$

On input $(STEALPWDFILE, sid, ID_C)$ from $\mathcal{A}$:

- If there exists $\langle MFILE, ID_C, ID_S, x \rangle$, then return x to the $\mathcal{A}$
- Otherwise, corrupt the server, generate a random value x' for existing ID_C and return to the $\mathcal{A}$

On input $(NEWSESSION, sid, ID_C, ID_S, x, Client)$ from $\mathcal{A}$:

- Store a $(ID_C, ID_S, sid, x, Client)$ labeled FRESH, and send $(NEWSESSION, sid, ID_C, ID_S, Client)$ to $\mathcal{A}$

On input $(NEWSESSION, sid, ID_S, ID_C, Server)$ from $\mathcal{A}$:

- Retrieve record $(MFILE, ID_C, ID_S, x)$, and send $(NEWSESSION, sid, ID_S, ID_C, Server)$ to $\mathcal{A}$
- If a $(ID_C, ID_S, sid, x, Client)$ exists, then store $(ID_S, ID_C, sid, x, Server)$ labeled FRESH

On input $(COMPLETEAUTH, sid, ID_S, ID_C)$ from $\mathcal{A}$:

- If there is no record $(ID_S, ID_C, sid, pw, Server)$, retrieve record (sid, ID_C, ID_S, res) . If it does not exist, retrieve and check $(MFILE, ID_C, ID_S, [x])$ stored by $\mathcal{A}$
- If $res = 1$ then send SUCCESS to ID_S and $\mathcal{A}$, else send FAIL
- If $res = 1$ then retrieve $\langle VAULT, ID_C, ID_S, eVault, evk, esk \rangle$ and send $(RETR, eVault, evk, esk)$ to the $\mathcal{A}$

On input $(OFFLINETESTPWD, sid, x')$ from $\mathcal{A}$:

- If there is a record $\langle MFILE, ID_C, ID_S, x' \rangle$, then return CORRECT. Otherwise, return WRONG to the $\mathcal{A}$

On message $(INIT/RETR, eVault, evk, esk)$ from $\mathcal{A}$:

- Retrieve record $\langle VAULT, ID_C, ID_S, eVault, evk, esk \rangle$, if there do not exist such a record, then abort.
- If this is a RETR query, then send $(COMPLETERETRS, sid, ID_C, ID_S)$

On message $(eVault', evk', esk')$ from $\mathcal{A}$:

- Update the record $\langle VAULT, ID_C, ID_S, eVault', evk', esk' \rangle$. //The simulator maintains storage consistency.
- Retrieve $(INITC, sid, ID_C, ID_S)$, delete it, and send $(COMPLETEINITC, sid, ID_C, ID_S)$ to the $\mathcal{F}_{adM3PM-EA}$

Figure 8: Simulator for M3PM protocol of 1Password.

$$|\Pr[G_5] - \Pr[G_4]| \leq Q_{init} \cdot \text{Adv}_{D,AE}^{\text{ind-cpa}}(n) \quad (6)$$

Game G_6 : (Abort upon ciphertext forgery). In this game, the simulator keep a local storage for the consistency of $eVault, evk, esk$. Upon receiving INITC query, the simulator keep the storage $(VAULT, ID_C, ID_S, eVault, evk, esk)$. Therefore, the adversary cannot distinguish the game from the inconsistency of the encrypted storage or keys. If the simulator receives a new ciphertext created by adversary $\mathcal{A}$, then the simulator returns FAIL to the adversary.

The environment $\mathcal{Z}$ distinguishes G_5 and G_6 if $\mathcal{A}$ forges a ciphertext succeeding to decrypt by AE . Then, we reduce the distinguishability to the ciphertext integrity (INT-CTXT) of AE . If there exists an adversary $\mathcal{D}$ can break INT-CTXT of AE with probability $\text{Adv}_{D,AE}^{\text{int-ctxt}}(n)$, then the environment $\mathcal{Z}$ can use the $\mathcal{D}$ to distinguish G_6 and G_5

within probability $(Q_{init} + Q_{retr}) \cdot \text{Adv}_{D,AE}^{\text{int-ctxt}}(n)$ where Q_{init} and Q_{retr} are the times of initialization and retrieval.

$$|\Pr[G_6] - \Pr[G_5]| \leq (Q_{init} + Q_{retr}) \cdot \text{Adv}_{D,AE}^{\text{int-ctxt}}(n) \quad (7)$$

Game G_7 : (Use $\mathcal{F}_{M3PM-EA}$ functionality interface). In this game, we modify the simulator to interact with the $\mathcal{F}_{M3PM}$ through the required interface. This change cannot be distinguished by $\mathcal{Z}$, as it is internal. The final simulator is described in Fig. 8, and we use appropriate formal descriptions to balance accuracy and understandability.

$$\text{Ideal}_{\mathcal{Z}, SIM_{OP}, \mathcal{F}_{M3PM-EA}} = \Pr[G_7] = \Pr[G_6] \quad (8)$$

In conclusion, the advantage of environment $\mathcal{Z}$ in distinguishing between the real and ideal world is negligible. $\square$

Appendix D. Meta-Review

The following meta-review was prepared by the program committee for the 2025 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

D.1. Summary

The paper presents a security analysis of an existing selection of password managers (both industrial and academic) that use a master password to protect user passwords (either user-chosen or automatically generated) for different accounts.

D.2. Scientific Contributions

- Identifies an Impactful Vulnerability
- Provides a Valuable Step Forward in an Established Field

D.3. Reasons for Acceptance

- 1) The paper presents the first security analysis of the M3PM protocols used by modern PMs that use passwords to protect the password vault. The paper offers a systematic review of 43 password managers, with a classification based on the authentication method.
- 2) A definition of ideal functionality for those protocols. They heavily rely on previous works and only introduce minimal differences (e.g., the relaxed and address guessing variant).
- 3) The paper details a security analysis for password management protocols. It analyzes two significant protocols, 1Password and Passbolt, extremely well.

D.4. Noteworthy Concerns

- 1) The authors employed the Universal Composability (UC) for some sub-protocols while ignoring others. This assumes that the security analysis is preserved even when executing alongside other protocols.